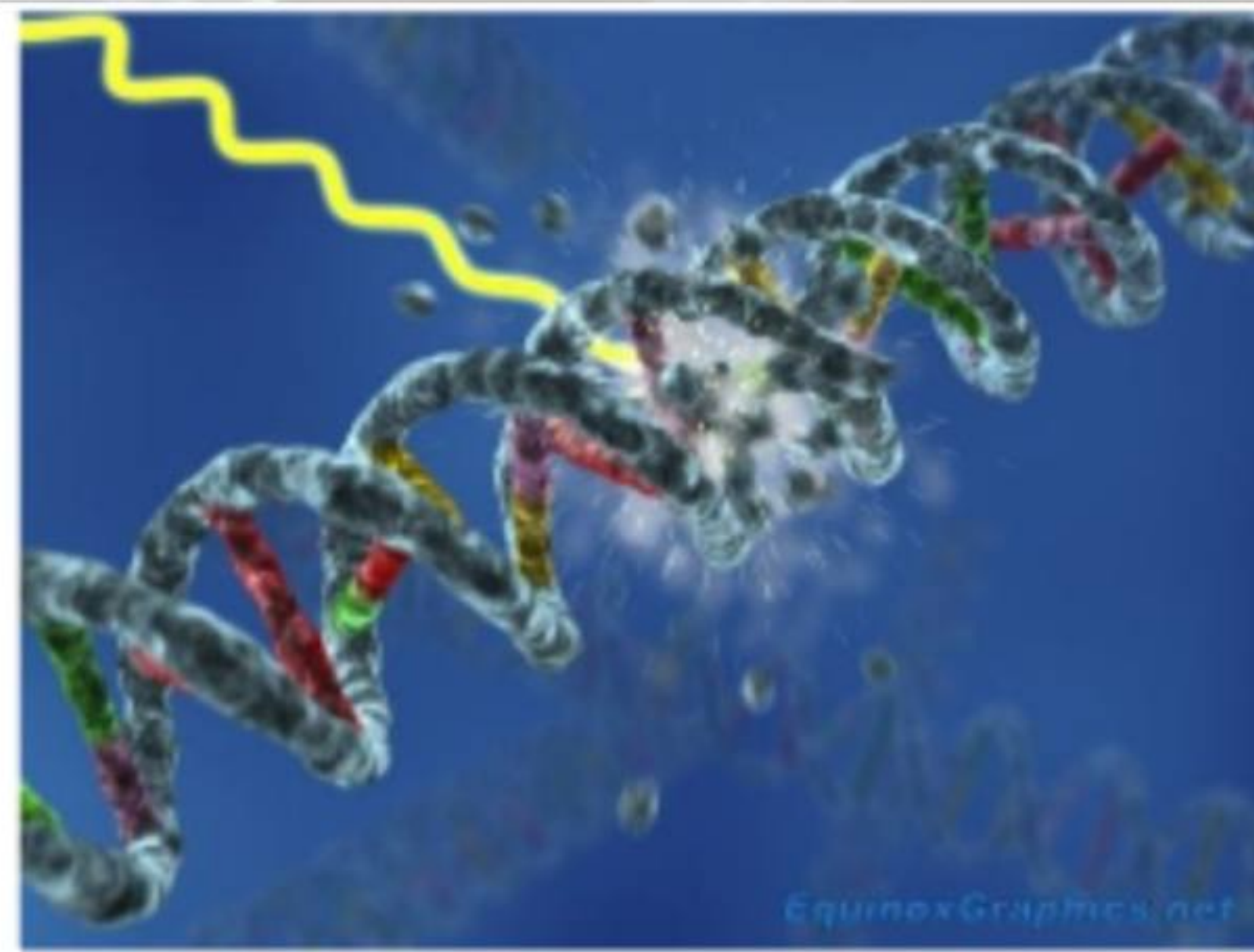


PLANIFICACIÓN DE TRATAMIENTOS DE RADIOTERAPIA

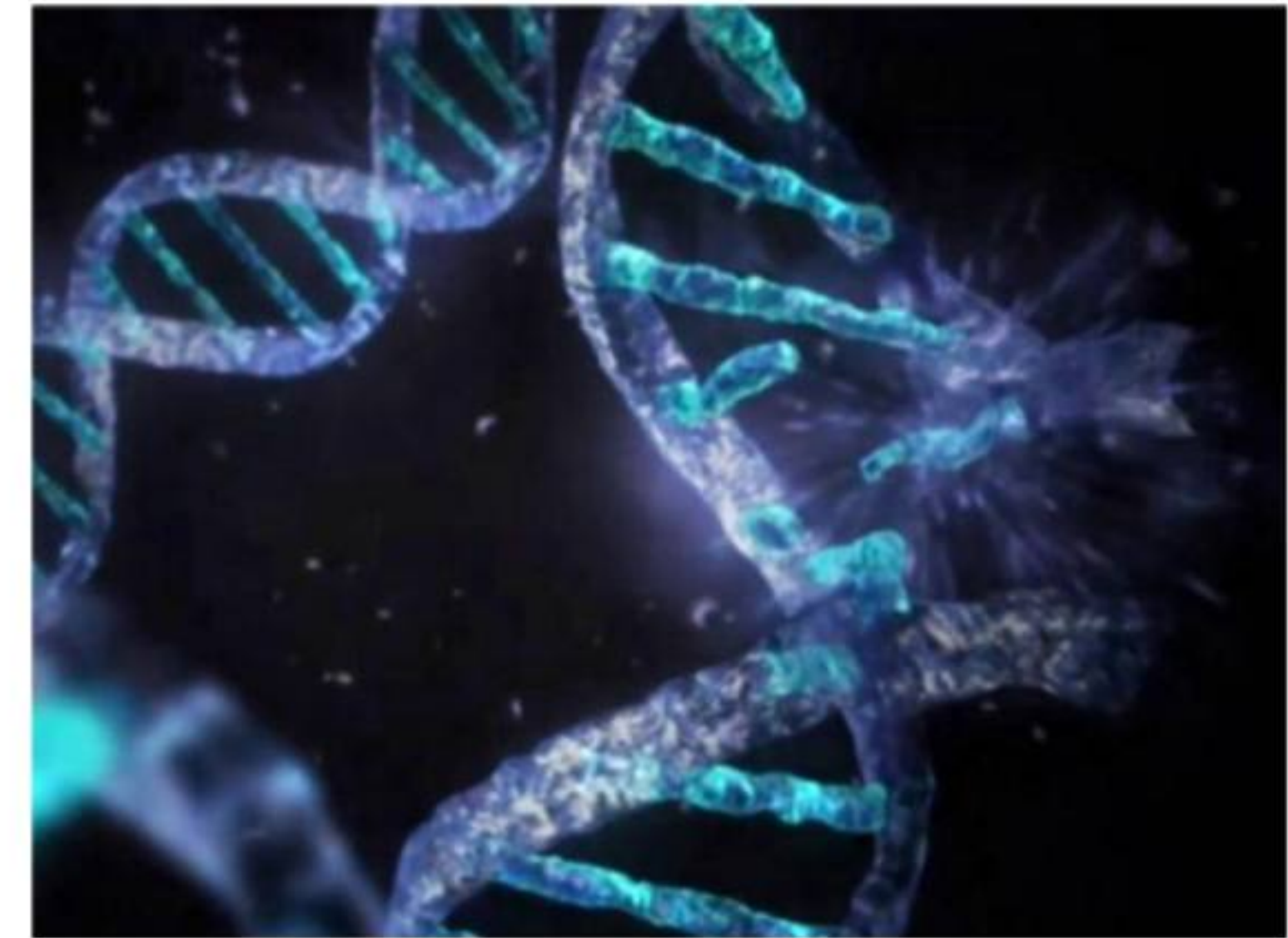


¿Qué son los tratamientos de radioterapia?

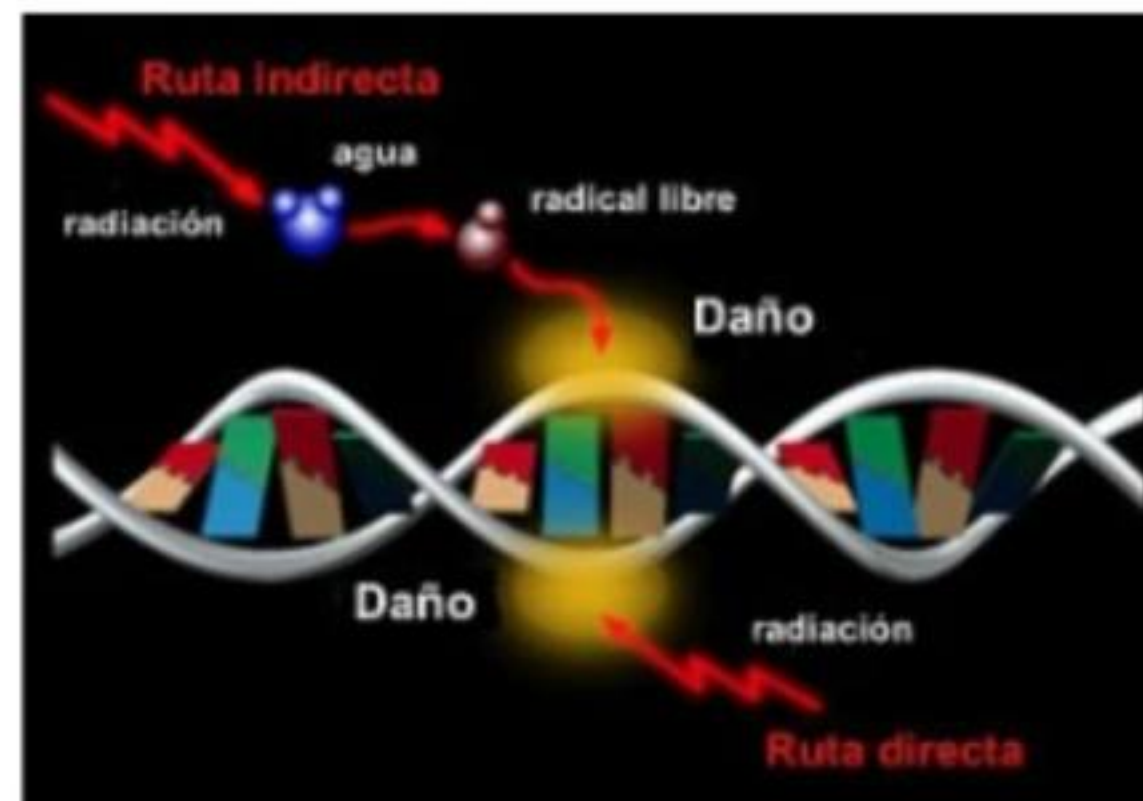
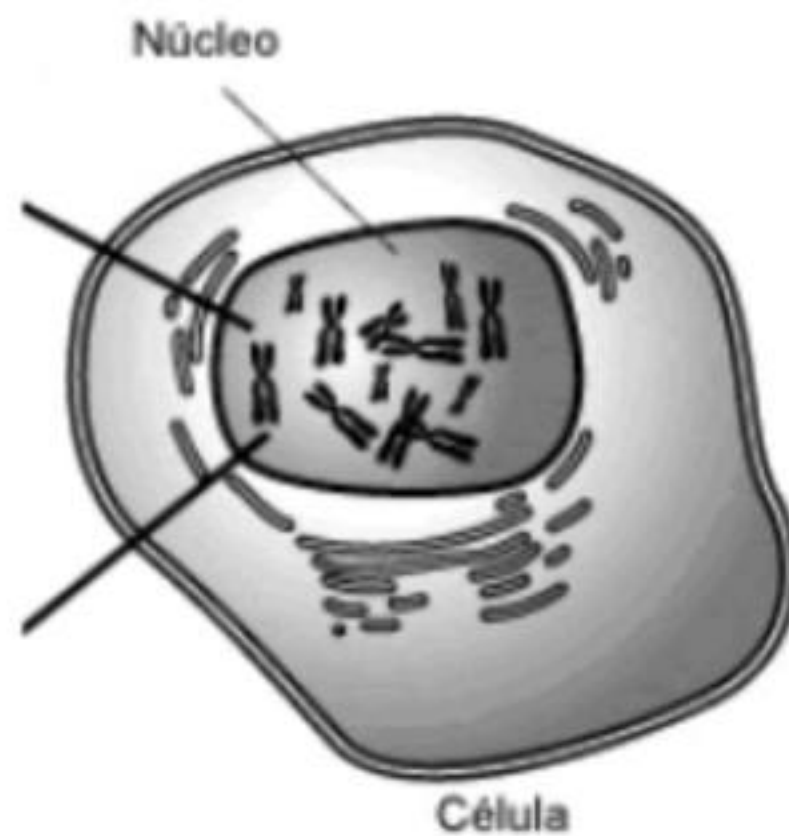
Ionización



Daños
directos



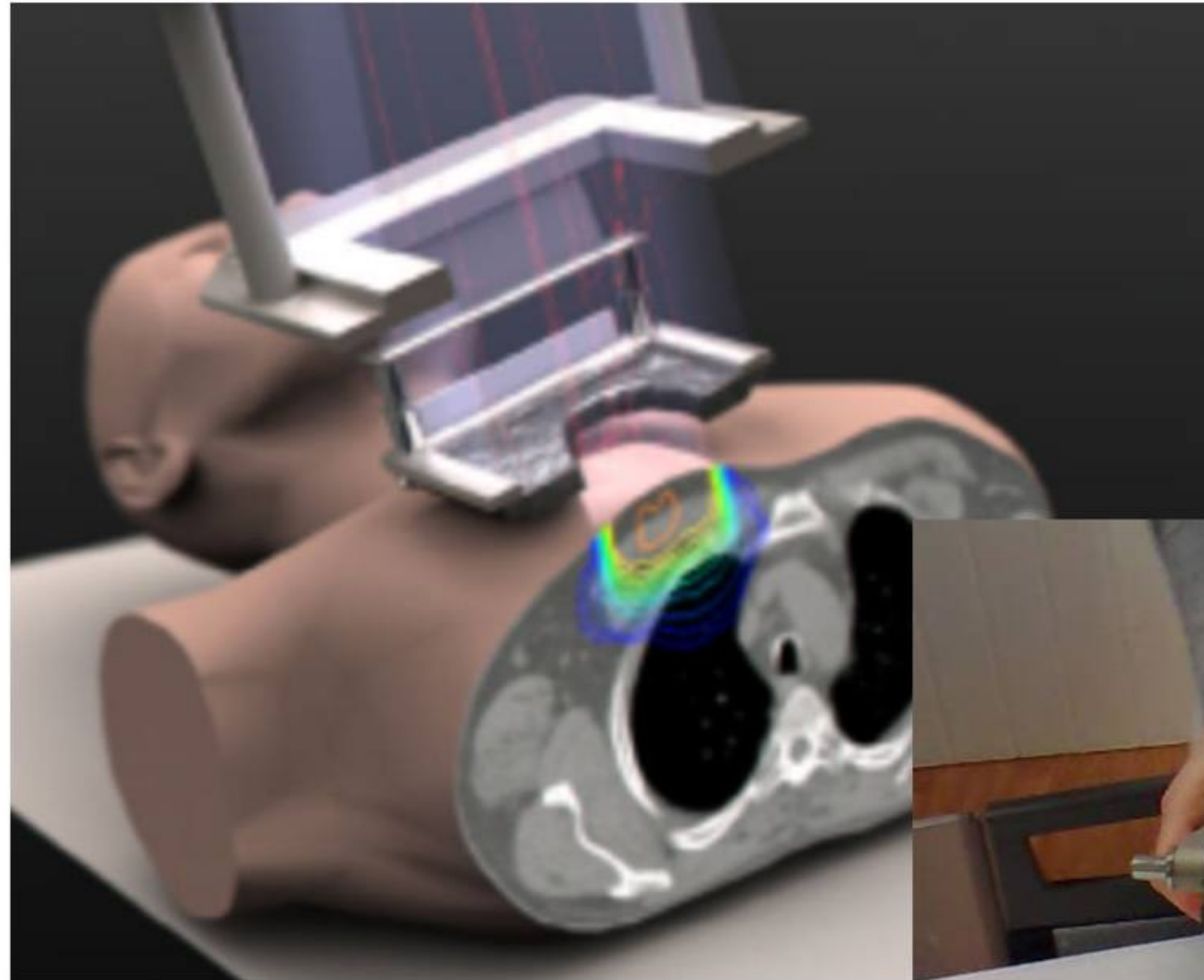
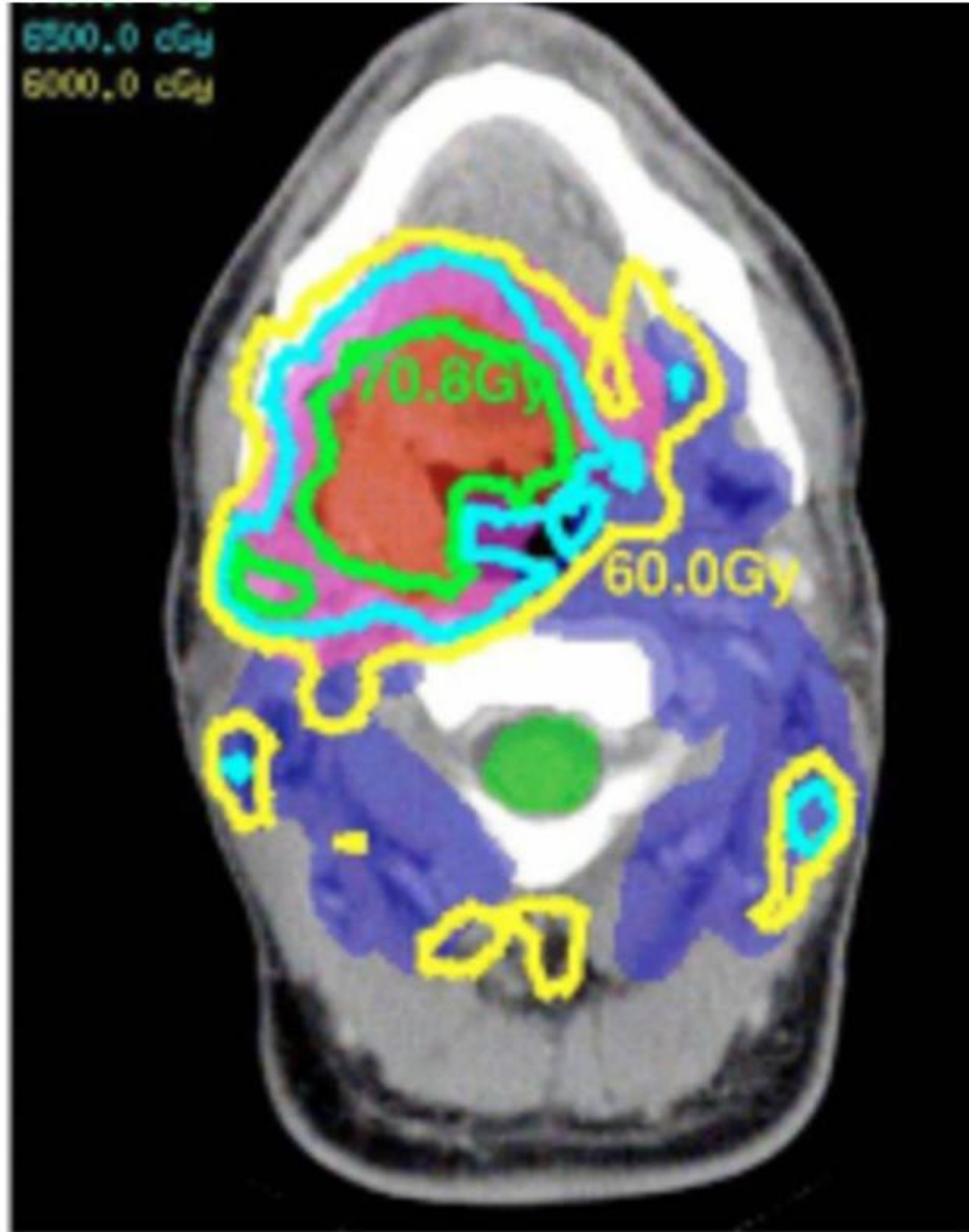
Daños
en las
células



Daños indirectos

¿Por qué es necesario simular los tratamientos?

Determinar la dosis absorbida



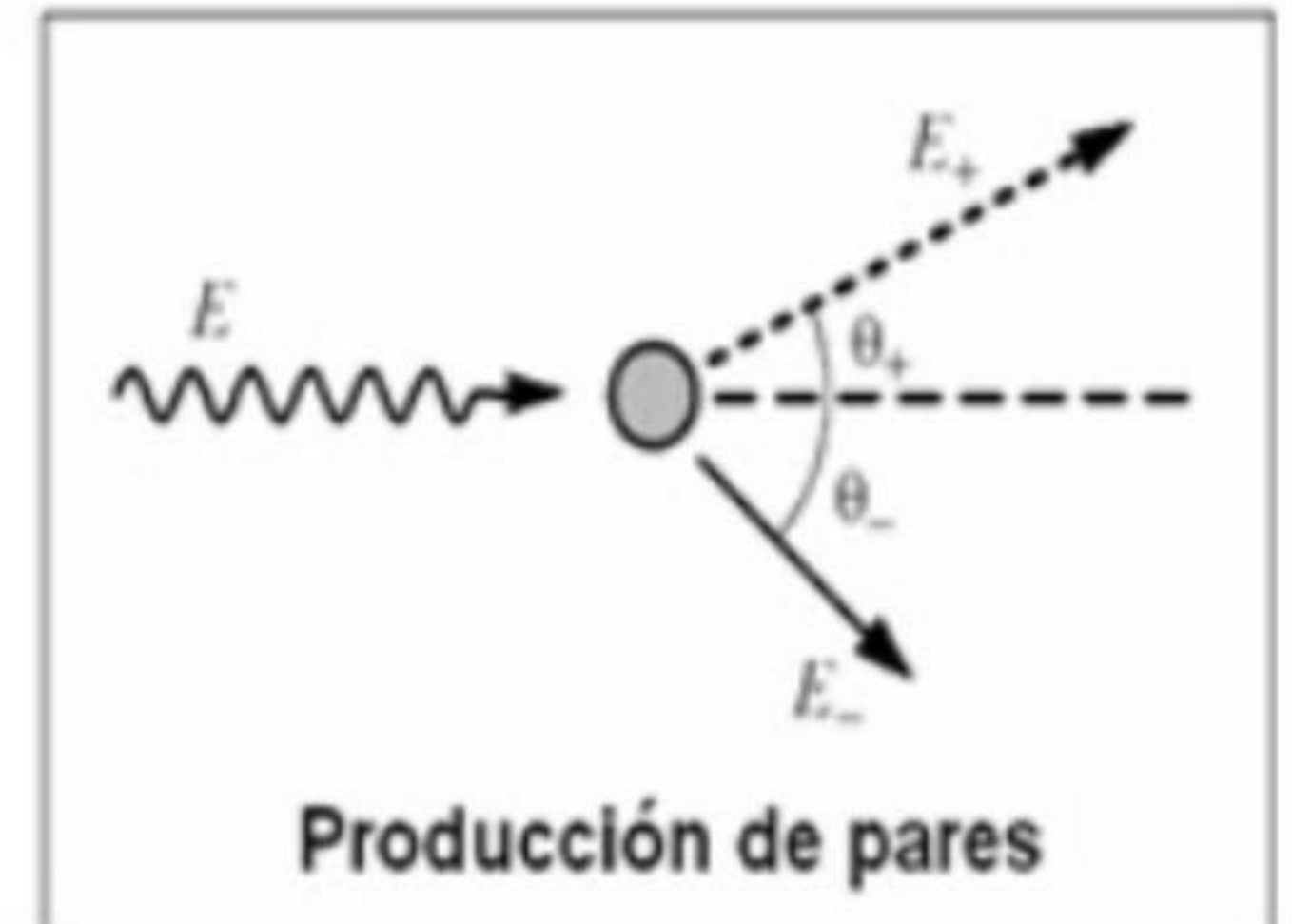
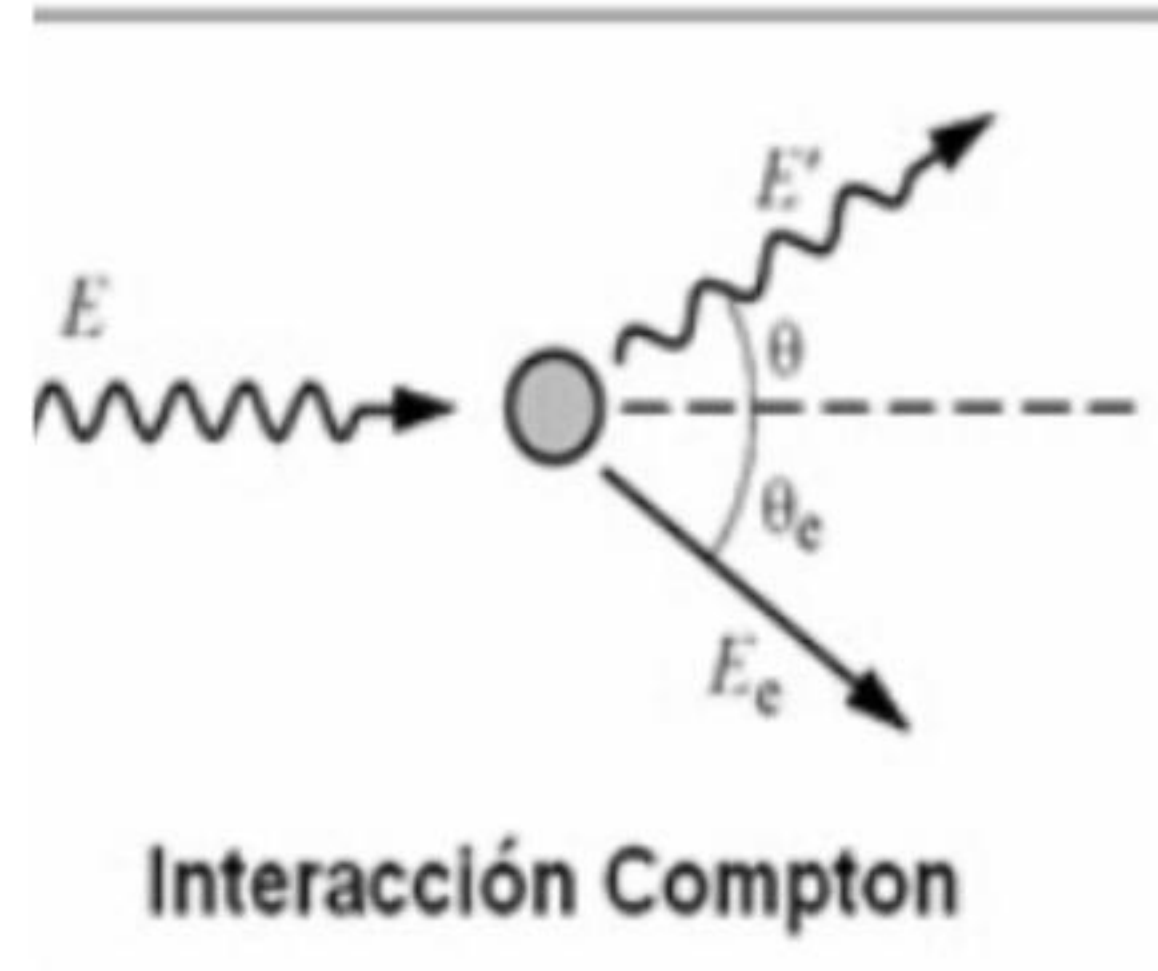
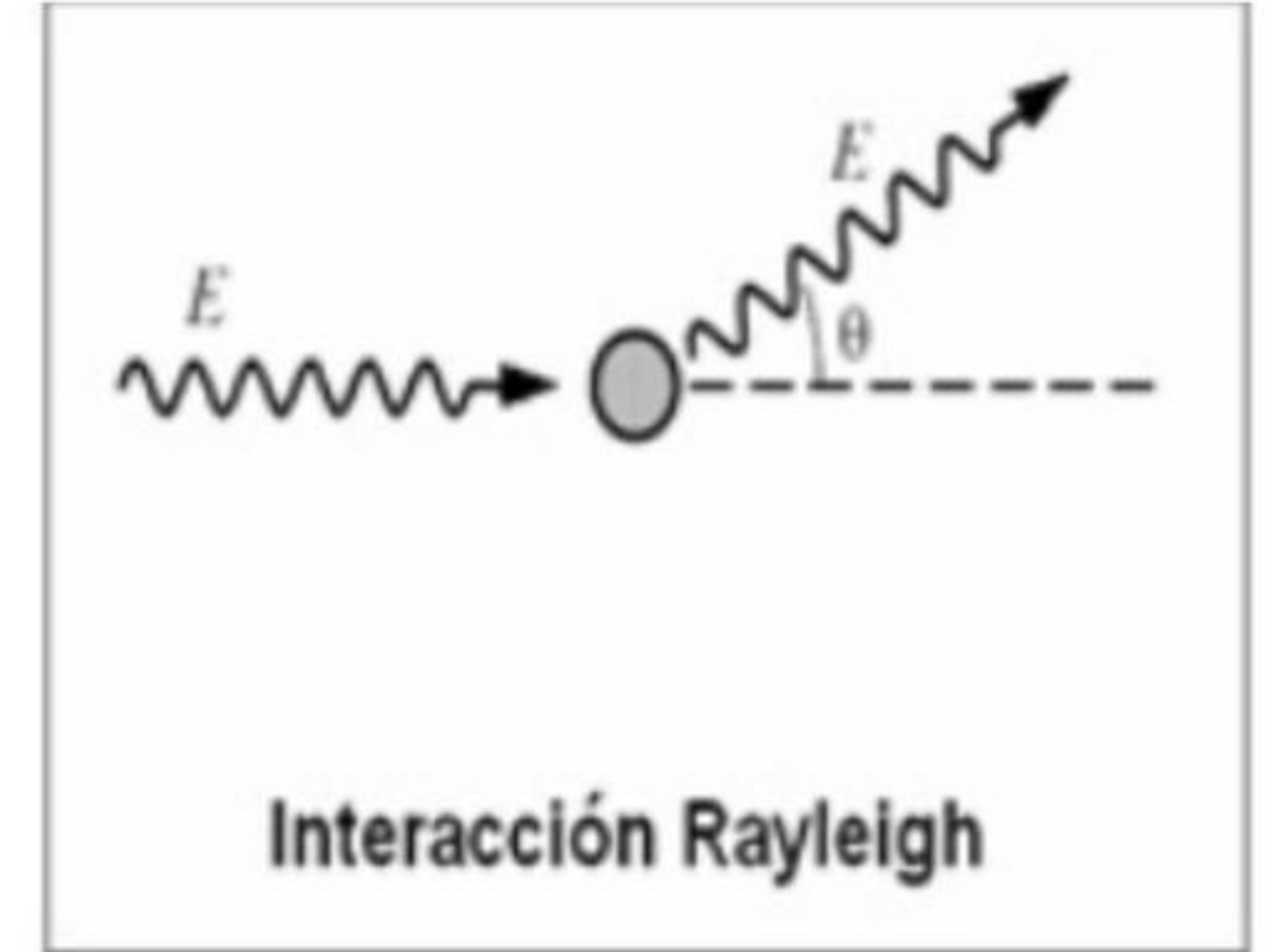
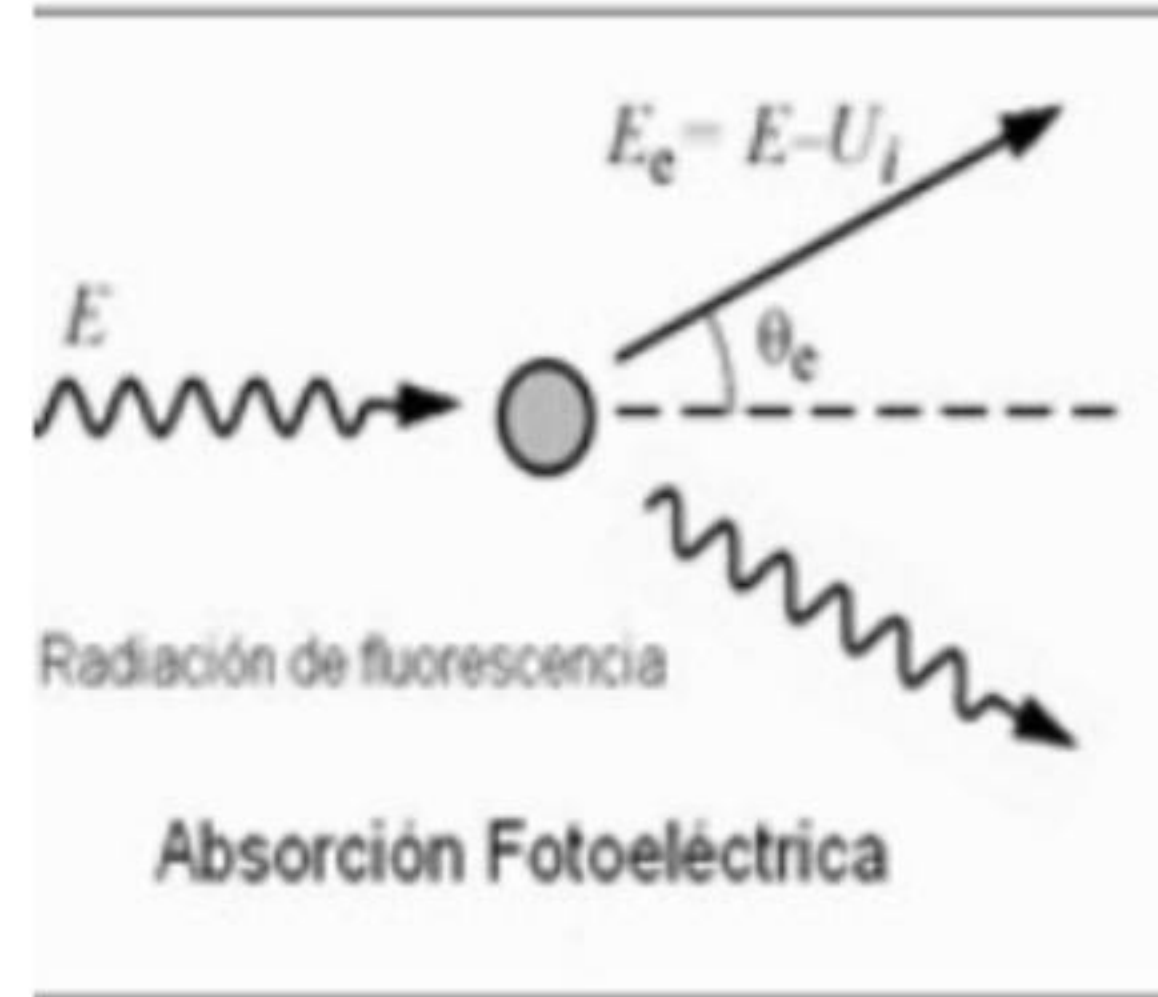
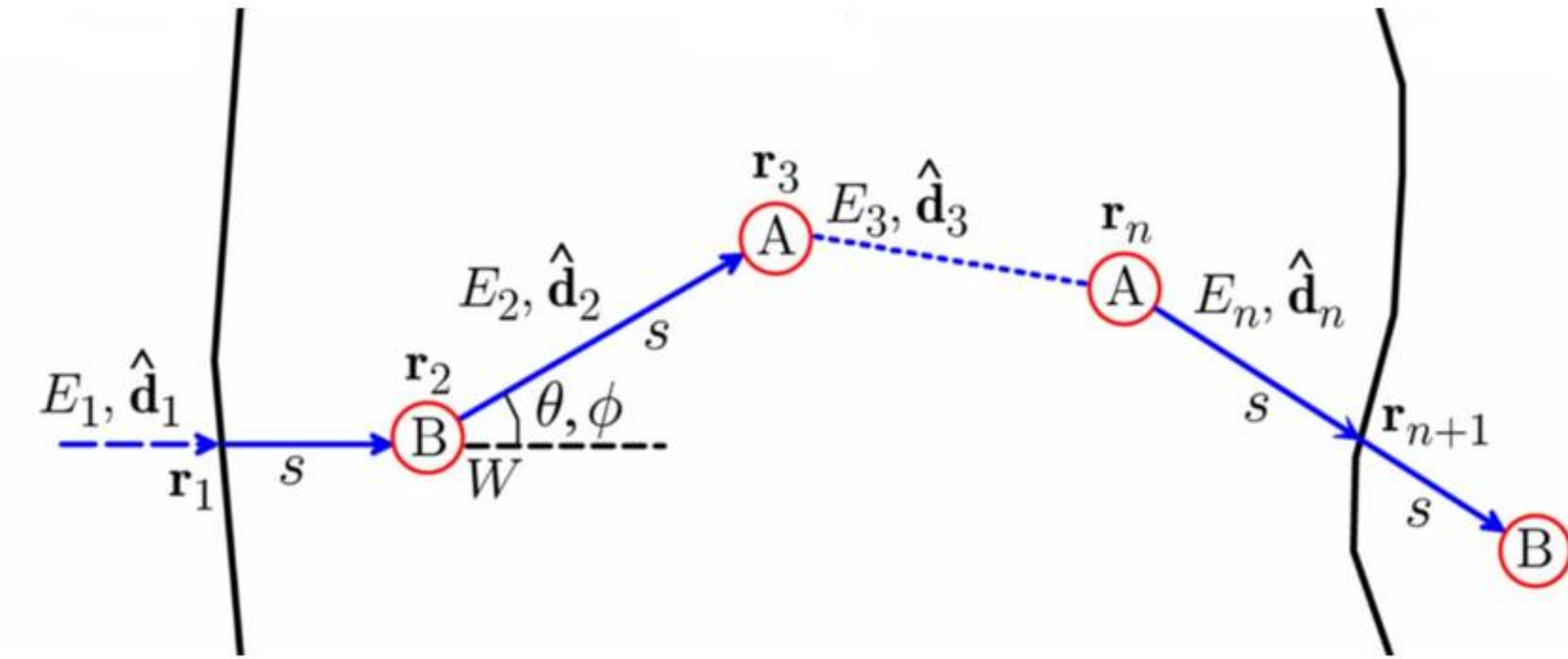
Fantomas
digitales

Fantomas
físicos



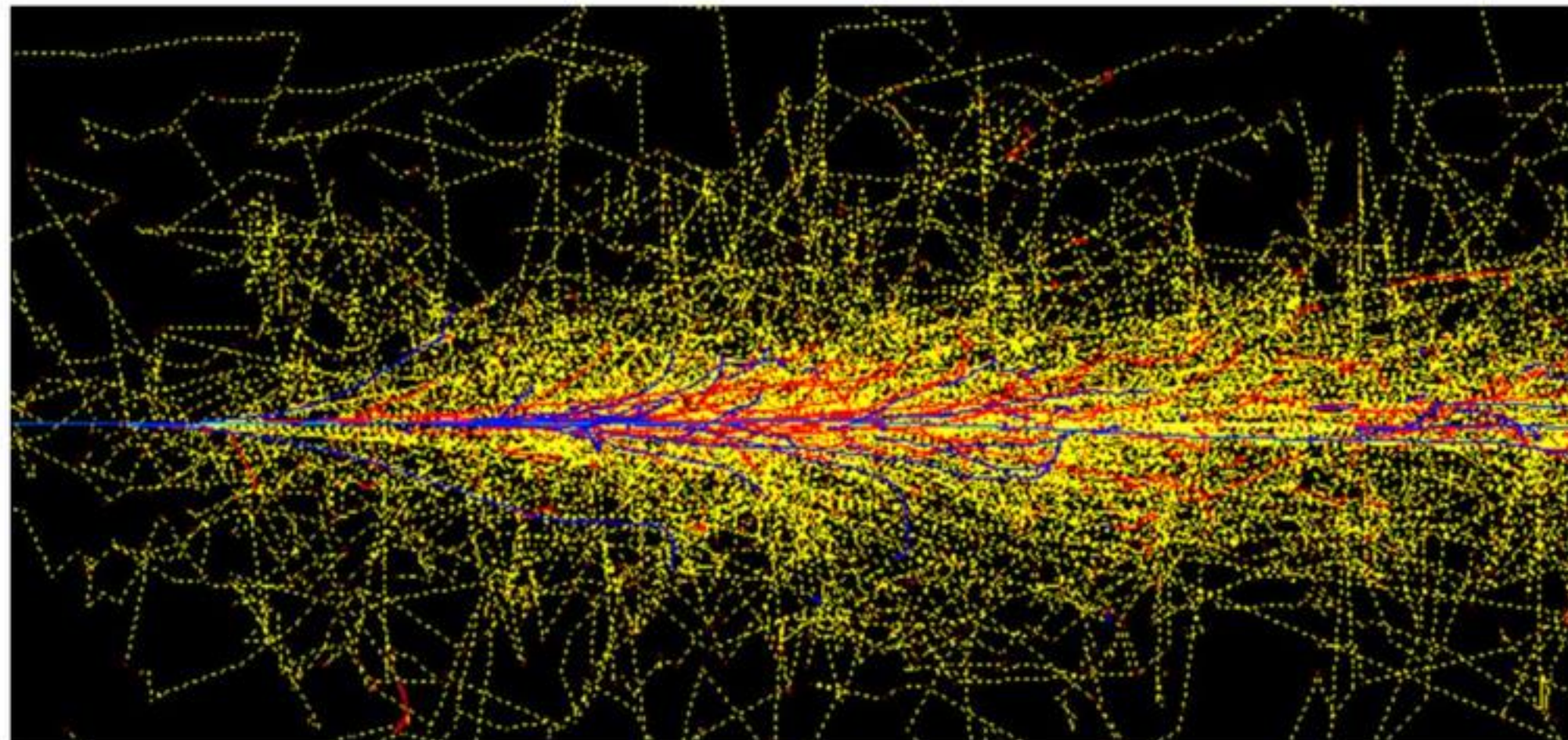
¿Cómo se simulan los tratamientos?

Método de Montecarlo



Interacciones de Fotones

- Seguir la pista de cada partícula
- Contar la energía depositada



¿Cuáles son las dificultades?

Modelar el paciente/medio



FOTÓN

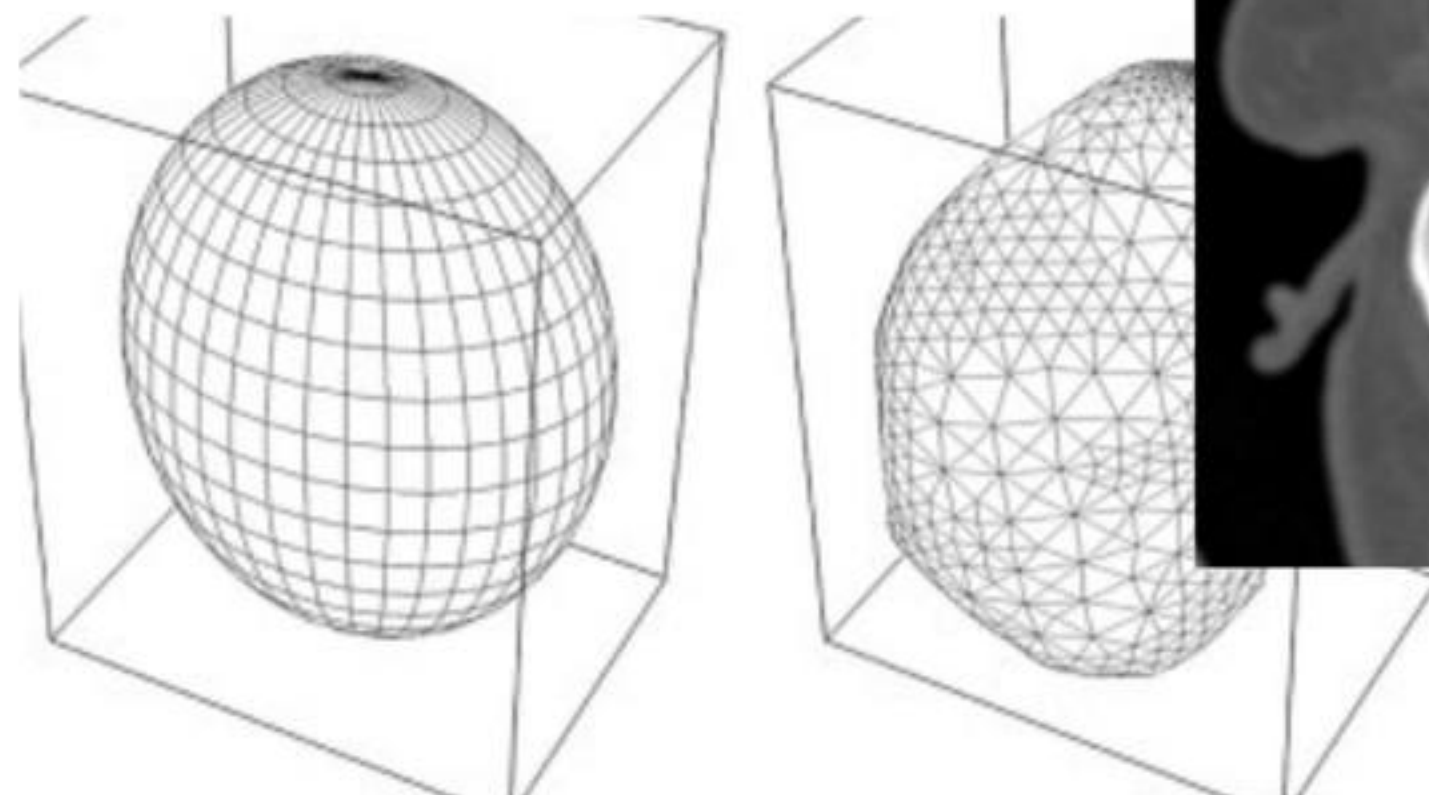
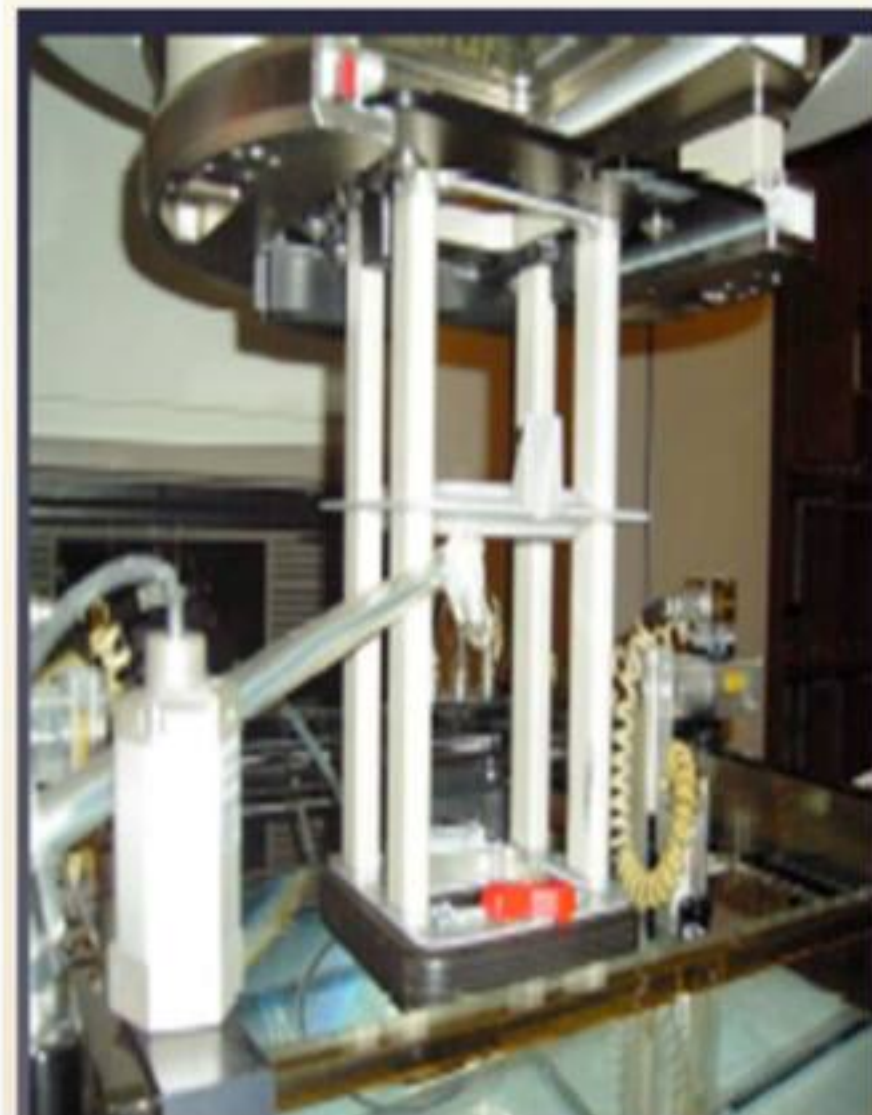


ELECTRÓN



POSITRÓN

Muchas! $\sim 10^9$

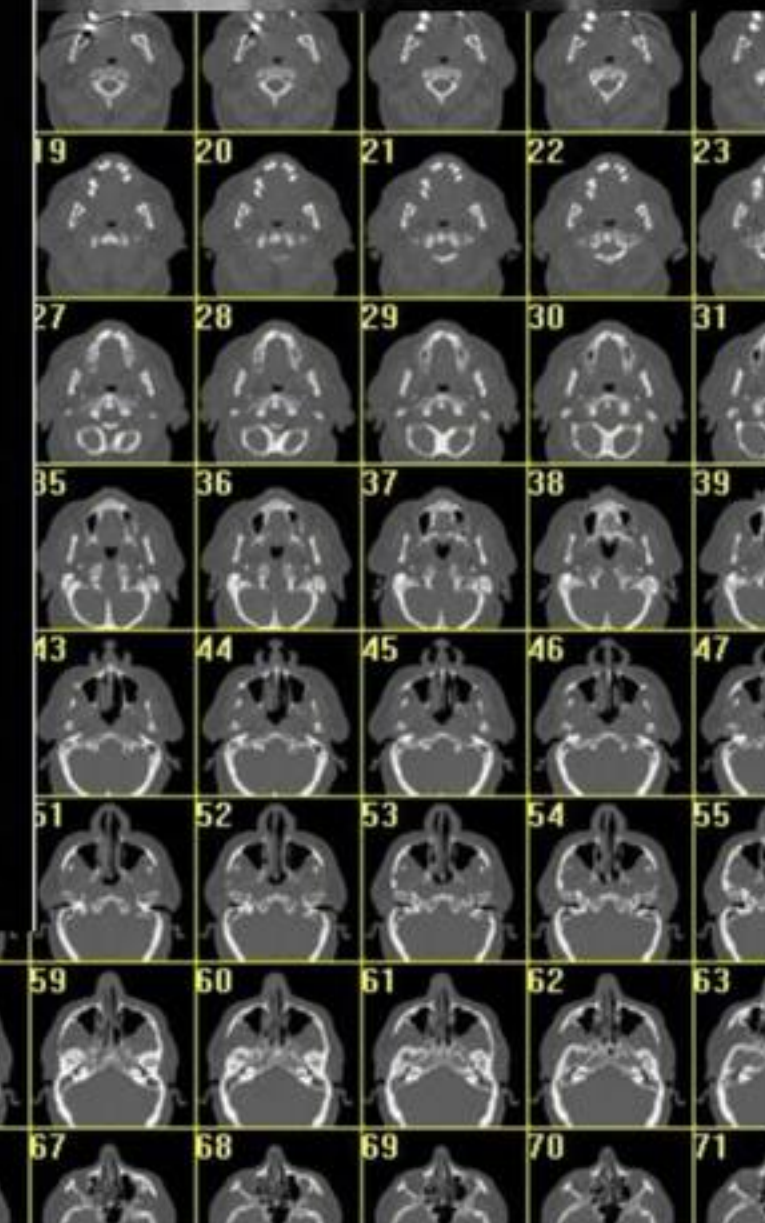


Modelar la fuente

```

000000000000000000000000000000000000
BODY ( 65) STOMACH CONTENTS
MATERIAL( 1)
SURFACE ( 104), SIDE POINTER=(-1)
000000000000000000000000000000000000
BODY ( 66) PANCREAS
MATERIAL( 5)
SURFACE ( 105), SIDE POINTER
SURFACE ( 100), SIDE POINTER
SURFACE ( 100), SIDE POINTER
000000000000000000000000000000000000

```



¿Cuáles problemas tiene el cálculo?

- **Tiempo: ~4 meses para una simulación simple**
- **Precisión: Resolución del medio $\sim 10^{10}$ vóxels**

Optimización!

¿Cómo optimizar el cálculo sin perder precisión?

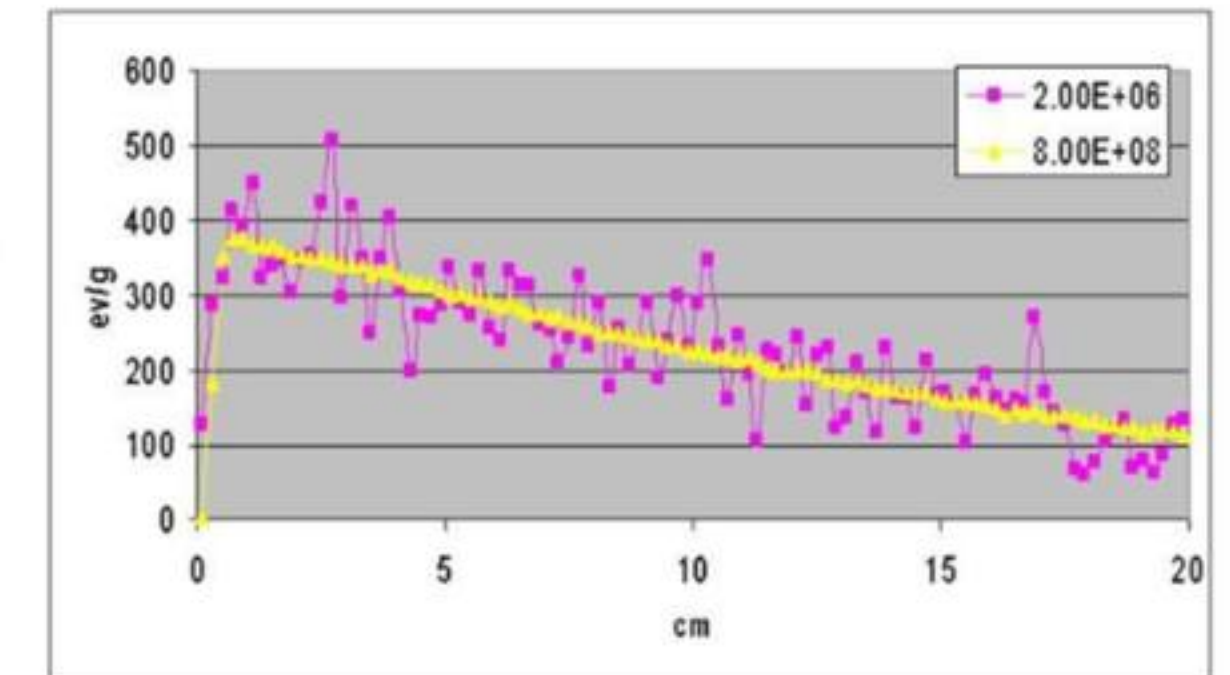
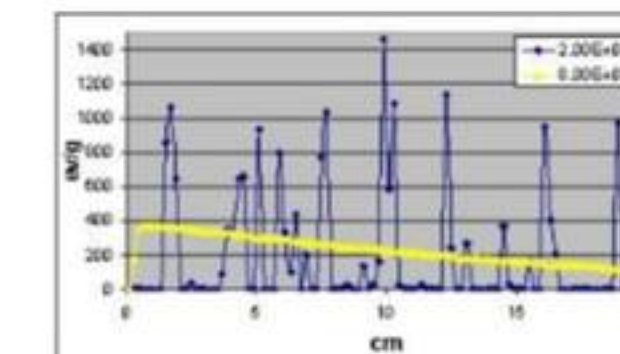
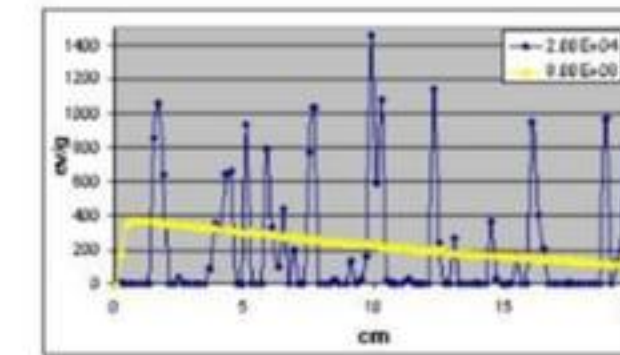
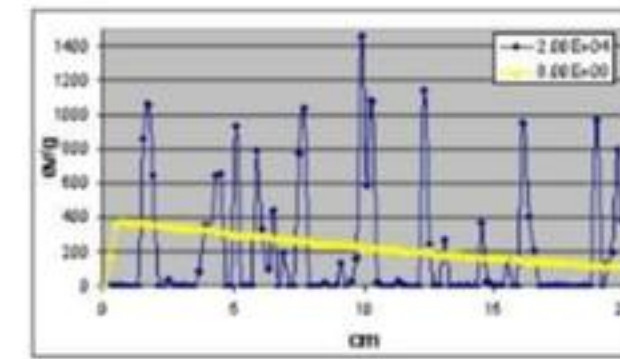
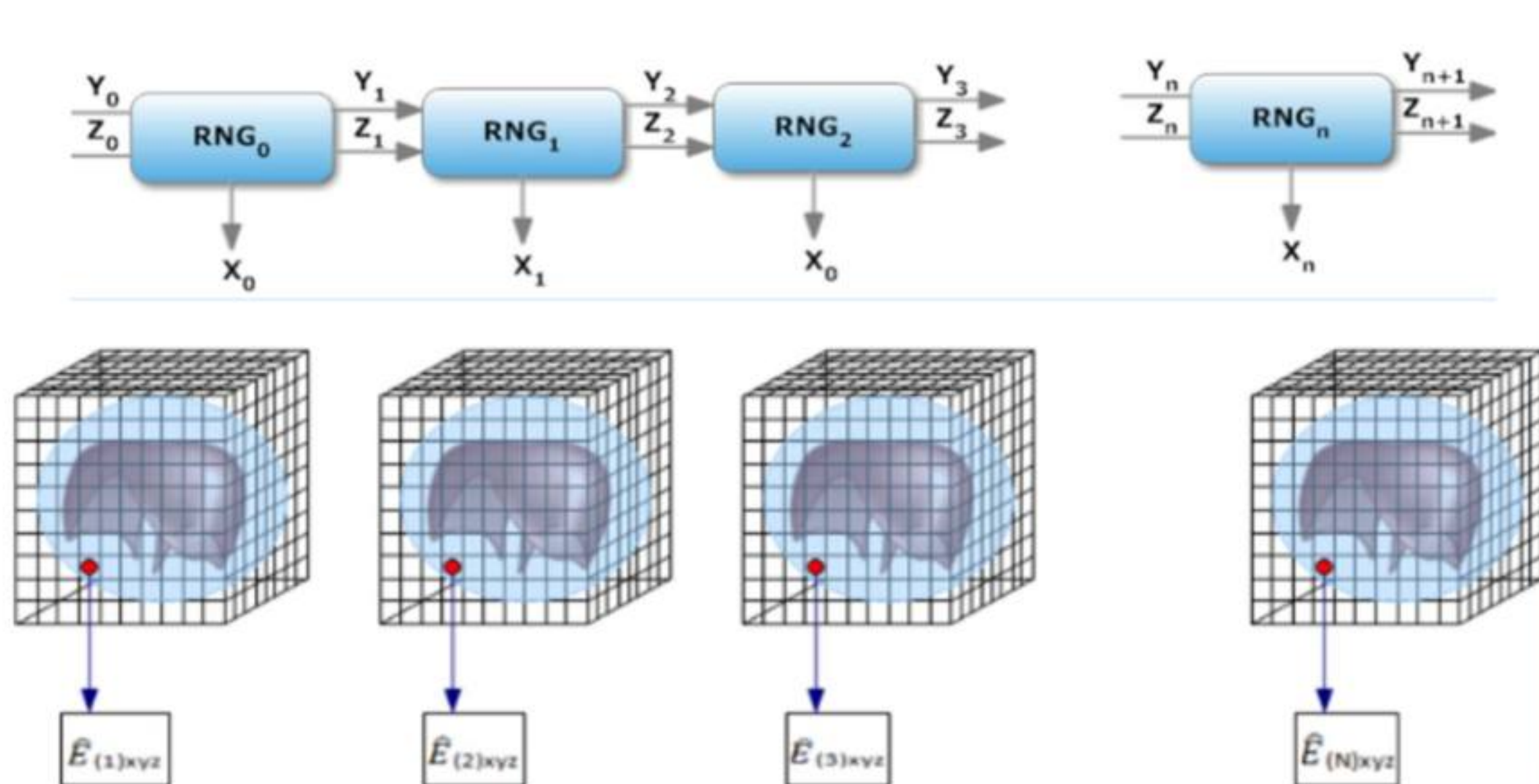
- **Reformulando las ecuaciones físicas**
- **Optimizando la complejidad computacional del algoritmo**
- **Optimizando la representación en memoria de la información**

Lenguaje de programación adecuado!

¿Cuáles son los lenguajes apropiados?

- Administración de memoria
- Concurrencia
- Generación de código

High Performance
FORTRAN



Múltiples hilos leyendo, calculando y escribiendo en matrices multidimensionales

¿Pueden usarse abstracciones?

Si, pero de costo cero



Puedo controlar
que el método no
sea virtual

```
class EstadoParticula {  
public:  
    double energia_ev;  
    std::array<double,3> posicion;  
    std::array<double,3> direccion;  
    double peso;  
    std::array<double,6> propiedades;  
    ...  
    double calcularinteraccion {  
        ...  
    }  
}
```

Puedo controlar
que los objetos
estén en el
segmento de
variables estáticas,
pila o heap

```
class EstadoFoton : EstadoParticula {  
public:  
    bool polarizado;  
    ...  
    double calcularinteraccion {  
        ...  
    }  
}  
int main() {  
    ...  
    EstadoParticula ep;  
    EstadoFoton ef;  
    ep = ef;  
    ...  
}
```

Un ejemplo sencillo

1. Definir una geometría

2. Para cada fotón de la fuente
(1 a $2 \cdot 10^5$)

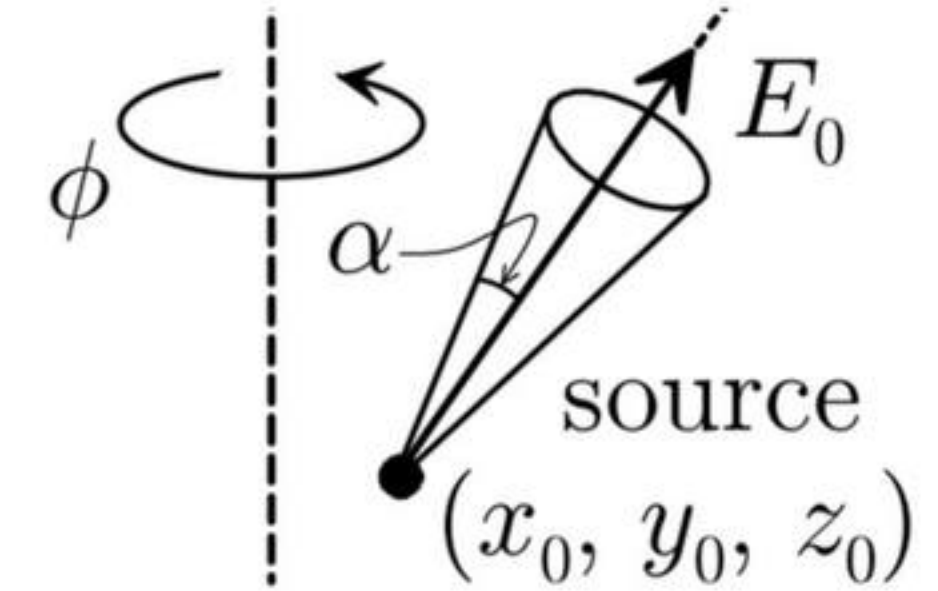
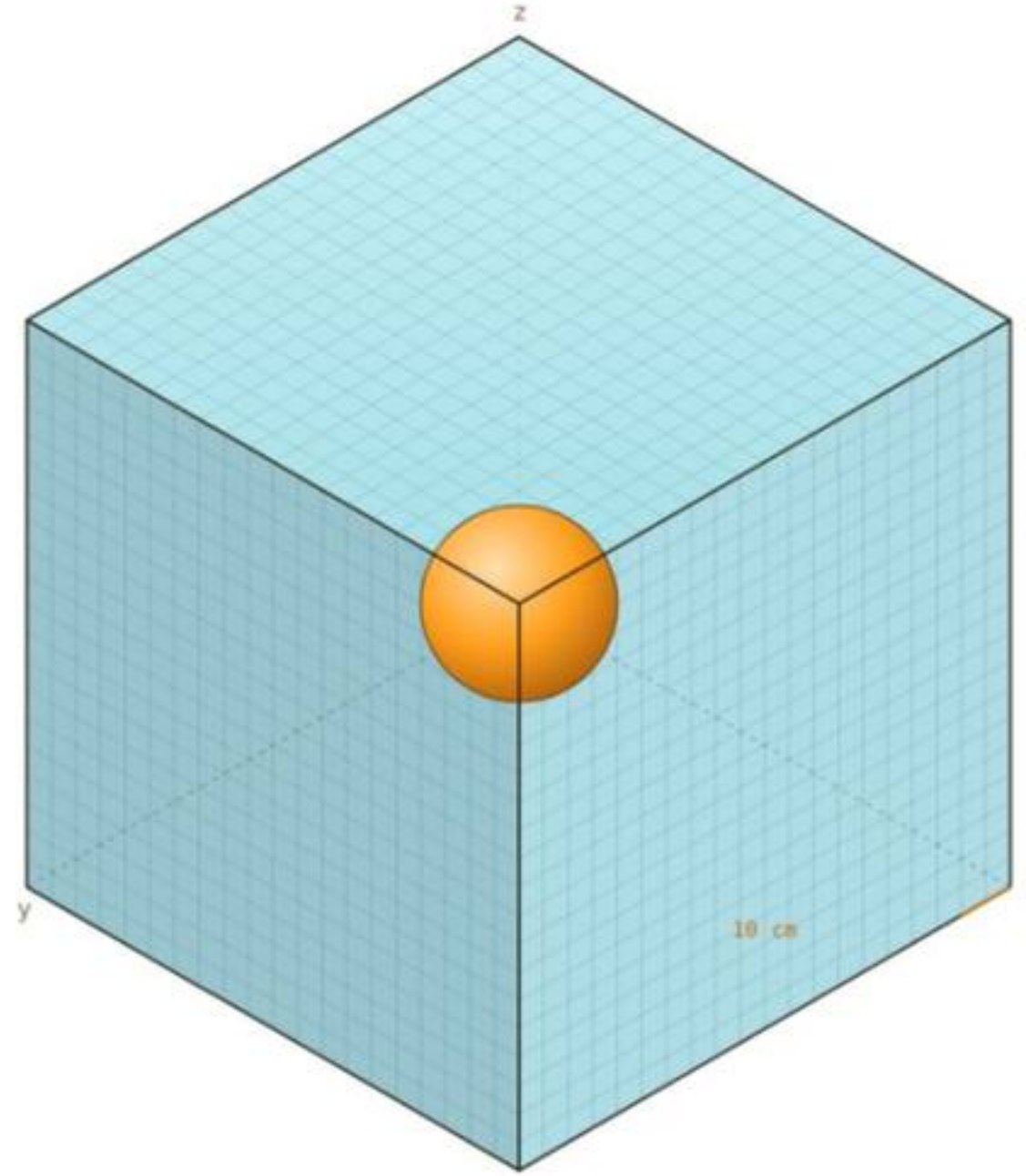
2.1 Muestrear dirección

2.2 Muestrear camino libre

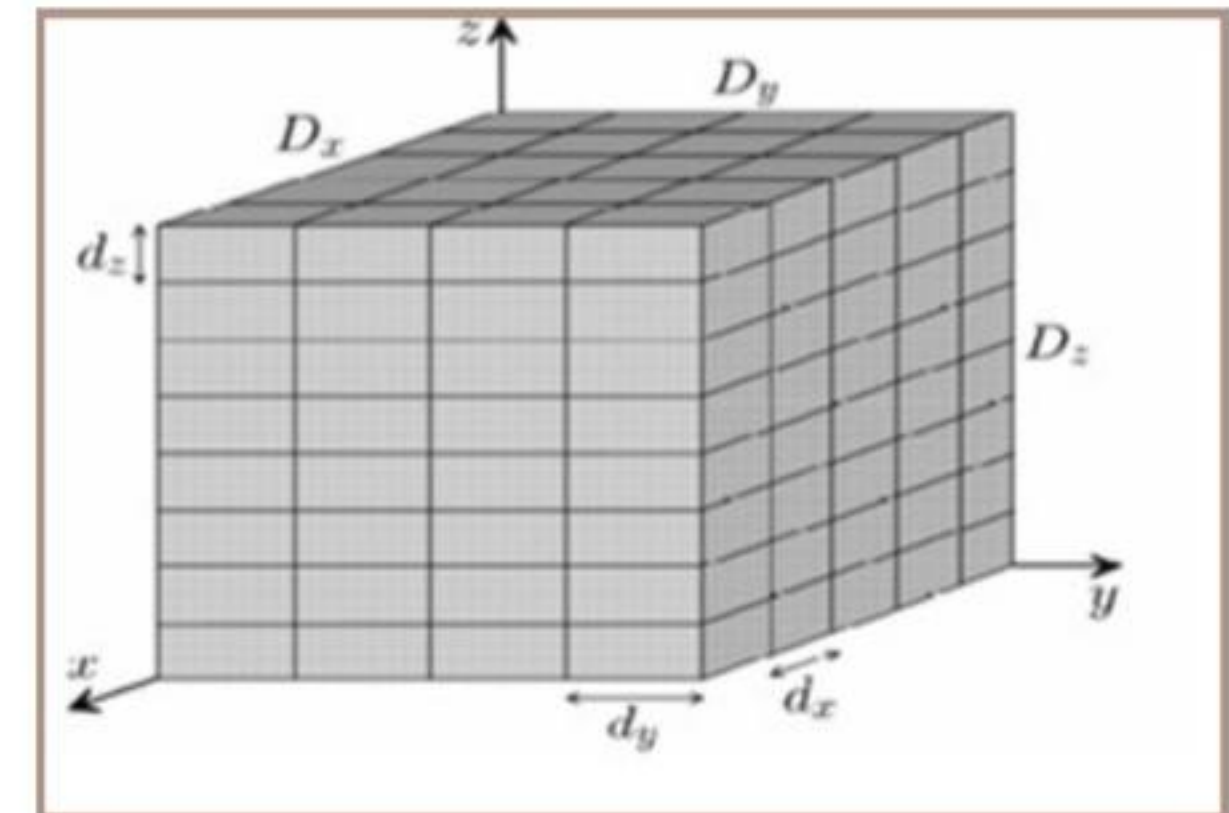
2.3 Recorrer voxeles de geometría según
dirección y camino

2.3.1 Si cambio el medio Volver a 2

3 Acumular estadísticas en matriz de resultados



$$s = -\lambda \ln(\xi)$$



Parámetros de la simulación

Parámetro	Valor
Dimensiones	20 × 20 × 20 vóxeles
Tamaño de vóxel	0.5 cm por lado
Dominio físico total	10 × 10 × 10 cm
Origen	(0, 0, 0)

Medio (coeficientes de absorción, a 100 keV)

Región	μ (1/cm)	Fuente
Fondo (agua)	0.1707	NIST/Attix, $\mu/\rho=0.1707$ cm ² /g, $\rho=1.00$ g/cm ³
Inclusión (hueso cortical)	0.3432	NIST/Attix, $\mu/\rho=0.1855$ cm ² /g, $\rho=1.85$ g/cm ³
Geometría de la inclusión	esfera de radio 4 vóxeles (2 cm), centrada	—

Fuente y muestreo

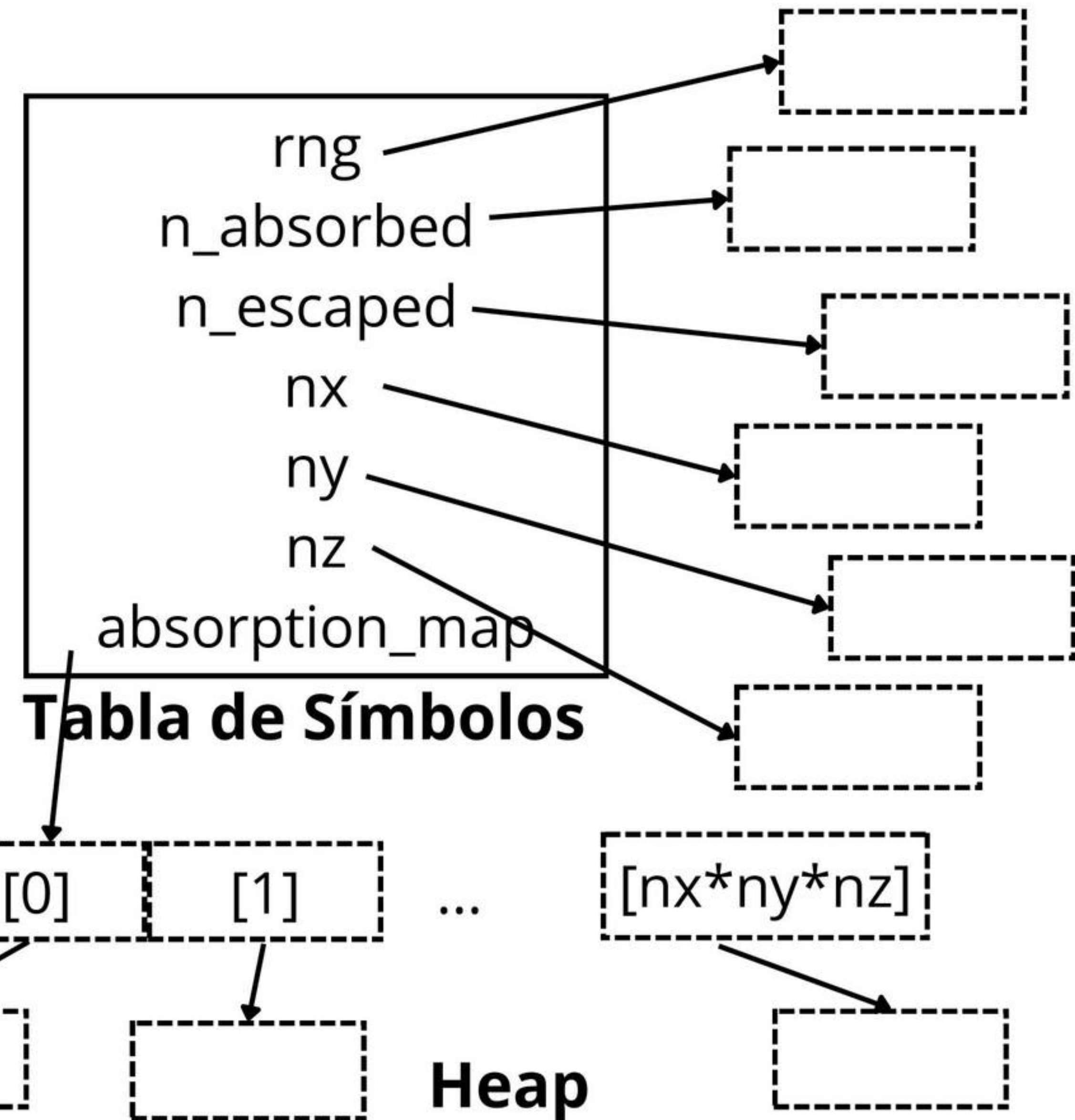
- **Energía del fotón:** 100 keV (fija, monoenergético)
 - **Posición de la fuente:** puntual, en el centro geométrico del dominio (5, 5, 5) cm
 - **Dirección:** isotrópica (uniforme sobre 4π estereorradianes), sorteada por fotón
 - **Camino libre:** por inversión de Beer-Lambert, $\tau = -\ln(\xi)$, acumulado vóxel a vóxel
-
- **Nº de fotones:** 200.000
 - **Épsilon de avance:** $1e-9 \times \text{tamaño_de_vóxel}$

¿Cómo implementarlo según el lenguaje?

Una implementación en Python



```
187 def run_simulation(grid: Grid3D, source_pos, n_photons, seed=0):
188     rng = Rng(seed)
189     n_absorbed = 0
190     n_escaped = 0
191     nx, ny, nz = grid.shape
192     absorption_map = [0] * (nx * ny * nz)
193
194     t_start = time.perf_counter()
195     for _ in range(n_photons):
196         absorbed, x, y, z = transport_photon(grid, source_pos, rng)
197         if absorbed:
198             n_absorbed += 1
199             idx = grid.voxel_index((x, y, z))
200             if grid.inside(idx):
201                 absorption_map[grid.flat_index(idx)] += 1
202         else:
203             n_escaped += 1
204     elapsed = time.perf_counter() - t_start
205
206     return {
```



Para crear cada variable:

- búsqueda de bloque libre en heap

Para acceder a cada valor:

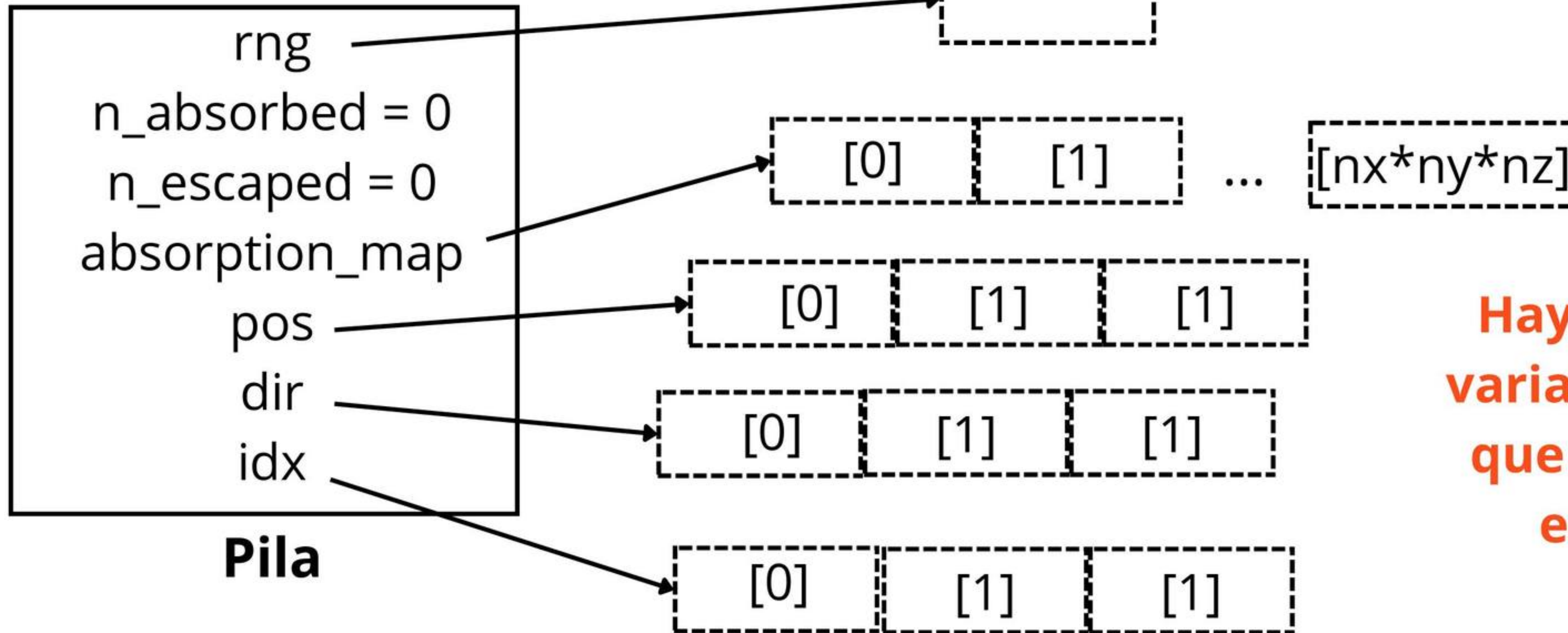
- búsqueda en TS
- 1 o 2 indirecciones en el heap

Una implementación en Java



```
191 static SimulationResult runSimulation(Grid3D grid, double[] sourcePos, int nPhotons, long seed) {  
192     Rng rng = new Rng(seed);  
193     int nAbsorbed = 0;  
194     int nEscaped = 0;  
195     long[] absorptionMap = new long[grid.shape[0] * grid.shape[1] * grid.shape[2]];  
196     double[] pos = new double[3];  
197     double[] dir = new double[3];  
198     int[] idx = new int[3];  
199 }
```

Heap



Hay ciertos tipos de variables que impiden que sean estáticas o estar en la pila

Una implementación en C



```
rng
n_absorbed = 0
n_escaped = 0
absorption_map
pos
dir
idx
```

Segmento estáticas o pila

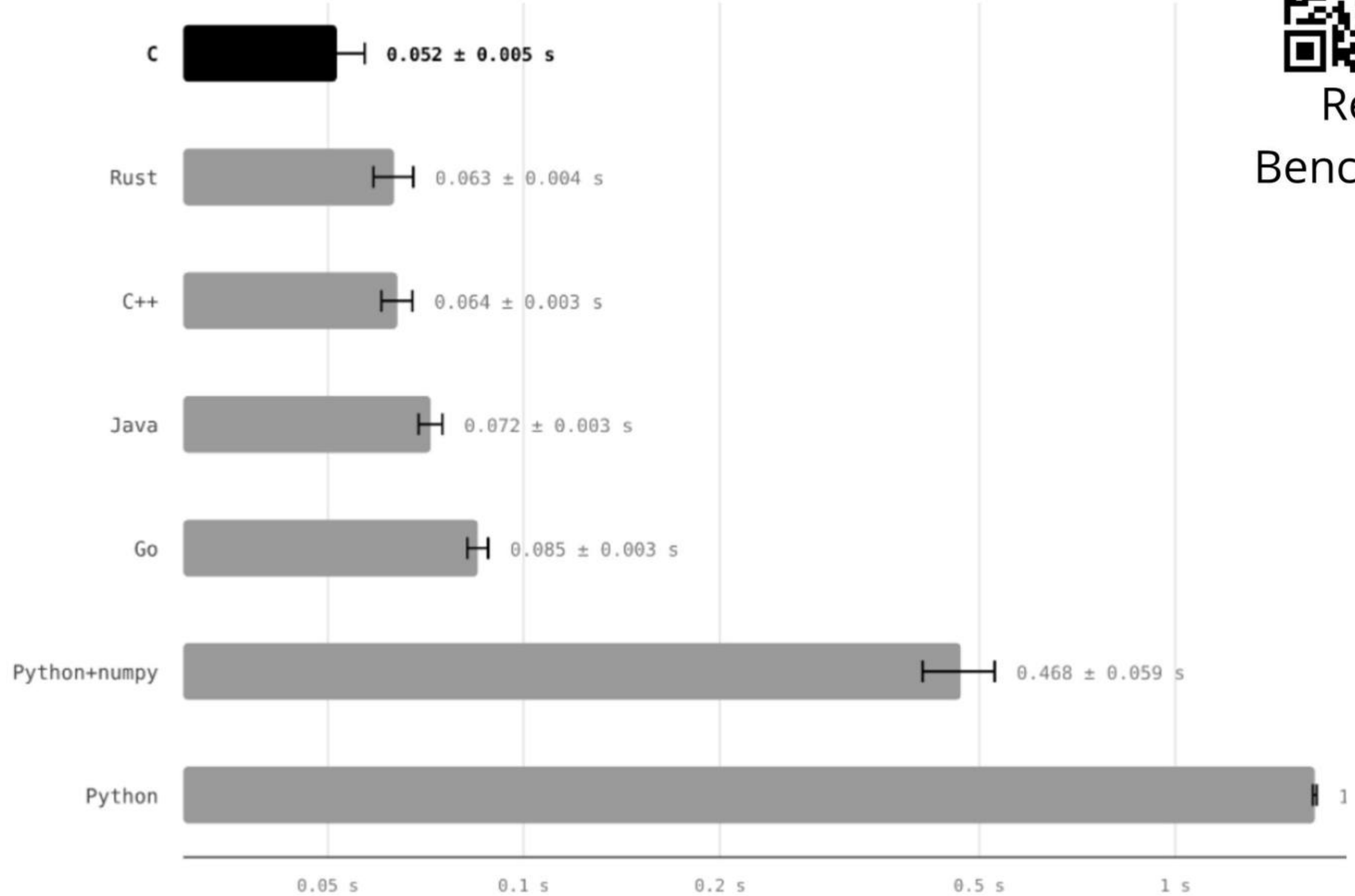
- La creación de las variables tienen costo cero
- El acceso a las variables tiene costo mínimo
- Puede evitarse por completo el uso del heap

**¿Cómo serían las
implementaciones
en otros lenguajes
(C++, Go, Rust)?**

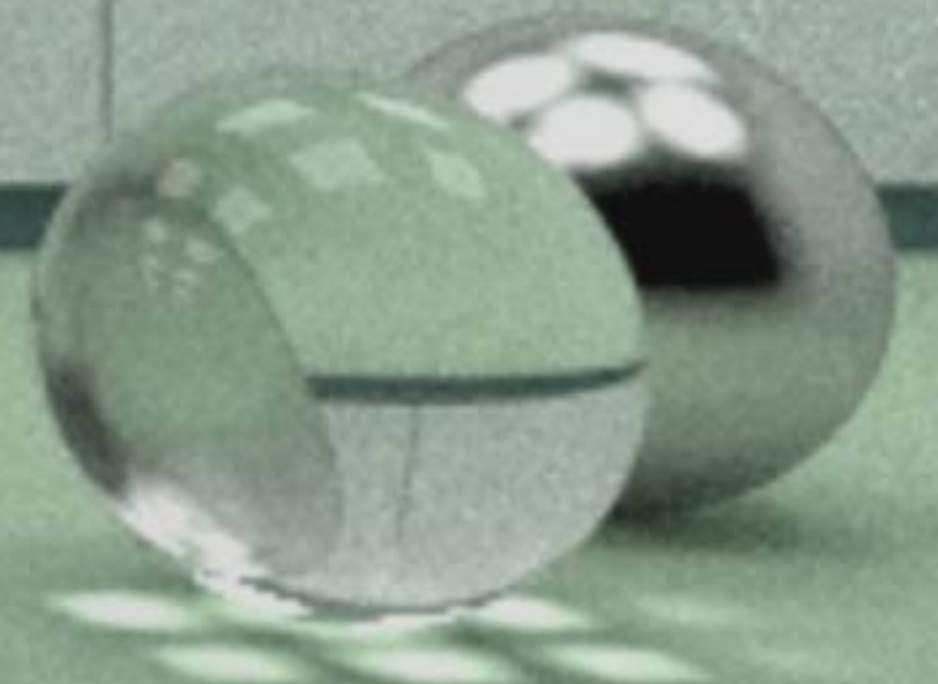
Todos contra todos



Repo
Benchmark



REWRITE RAY TRACER (it) IN **RUST**



¿Por qué esta idea?

Background

Autocad

Sketchup : Enscape & V-Ray

3ds Max

Blender



Volcar Conocimientos

Ownership & Borrowing

Lifetimes

Type safety

Smart Pointers

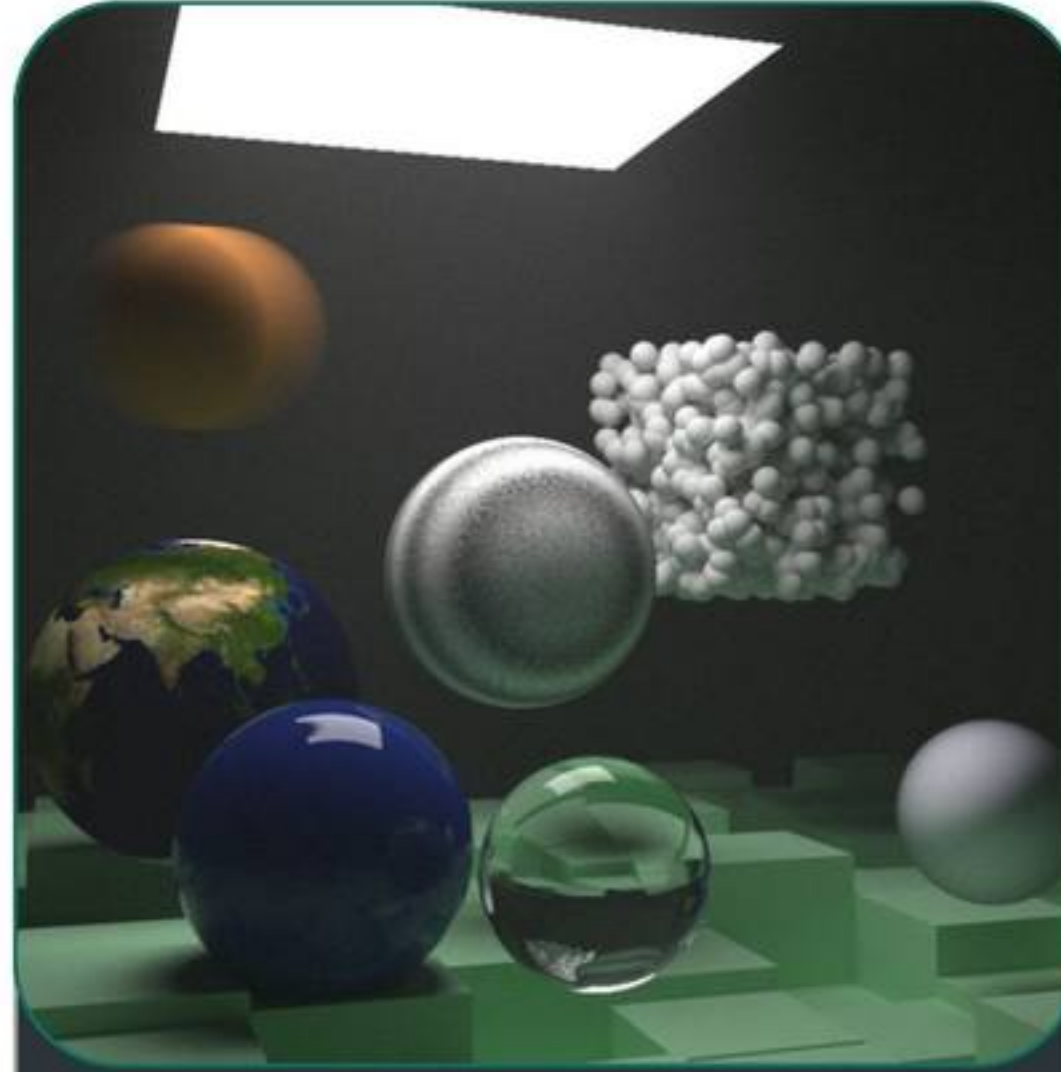
Concurrencia

The Book Series

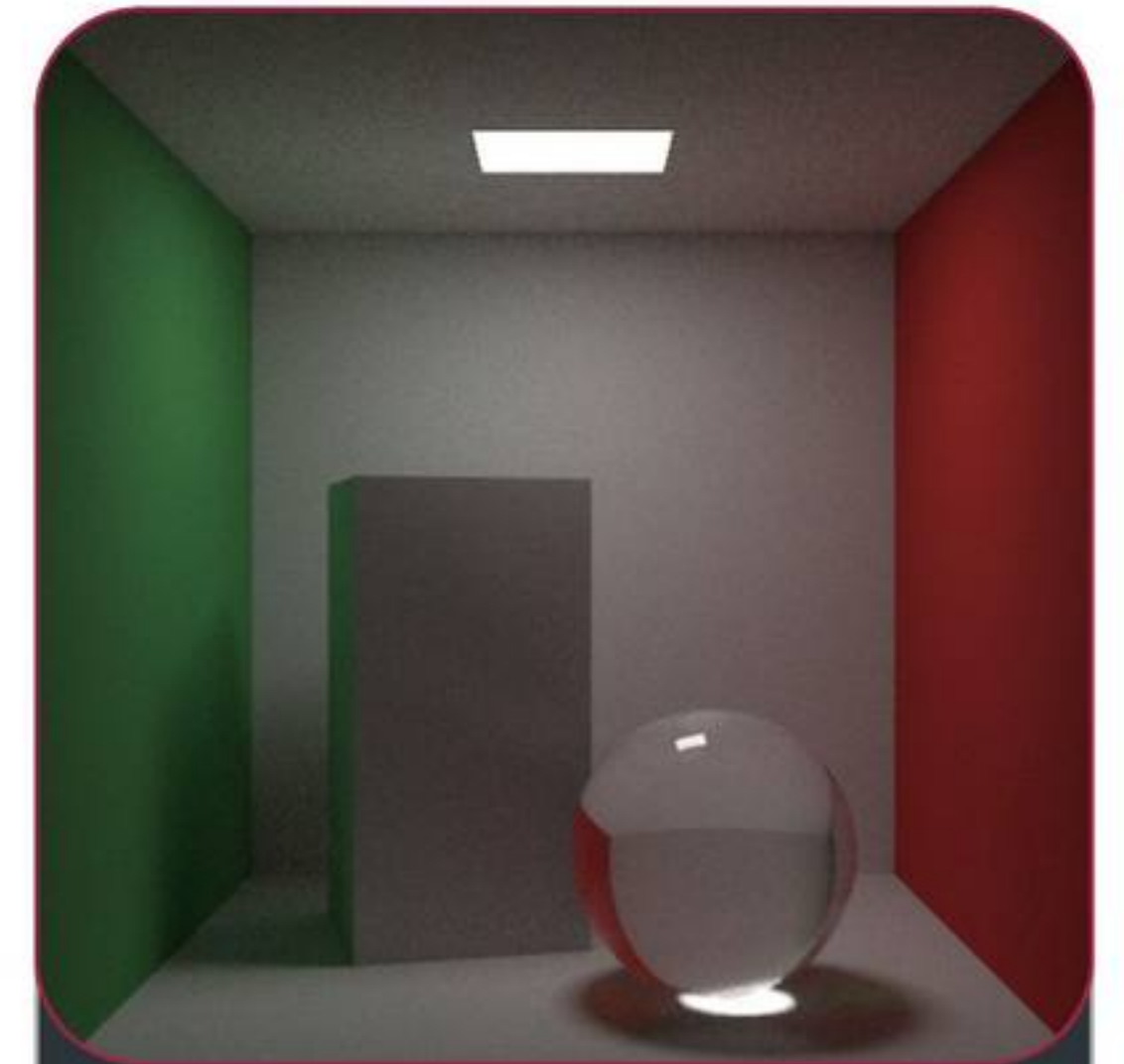
El proyecto se inspira directamente en la serie literaria de **Peter Shirley**, **Trevor David Black** y **Steve Hollasch**



Ray Tracing in One
Weekend



Ray Tracing: The
Next Week



Ray Tracing: The Rest
of Your Life

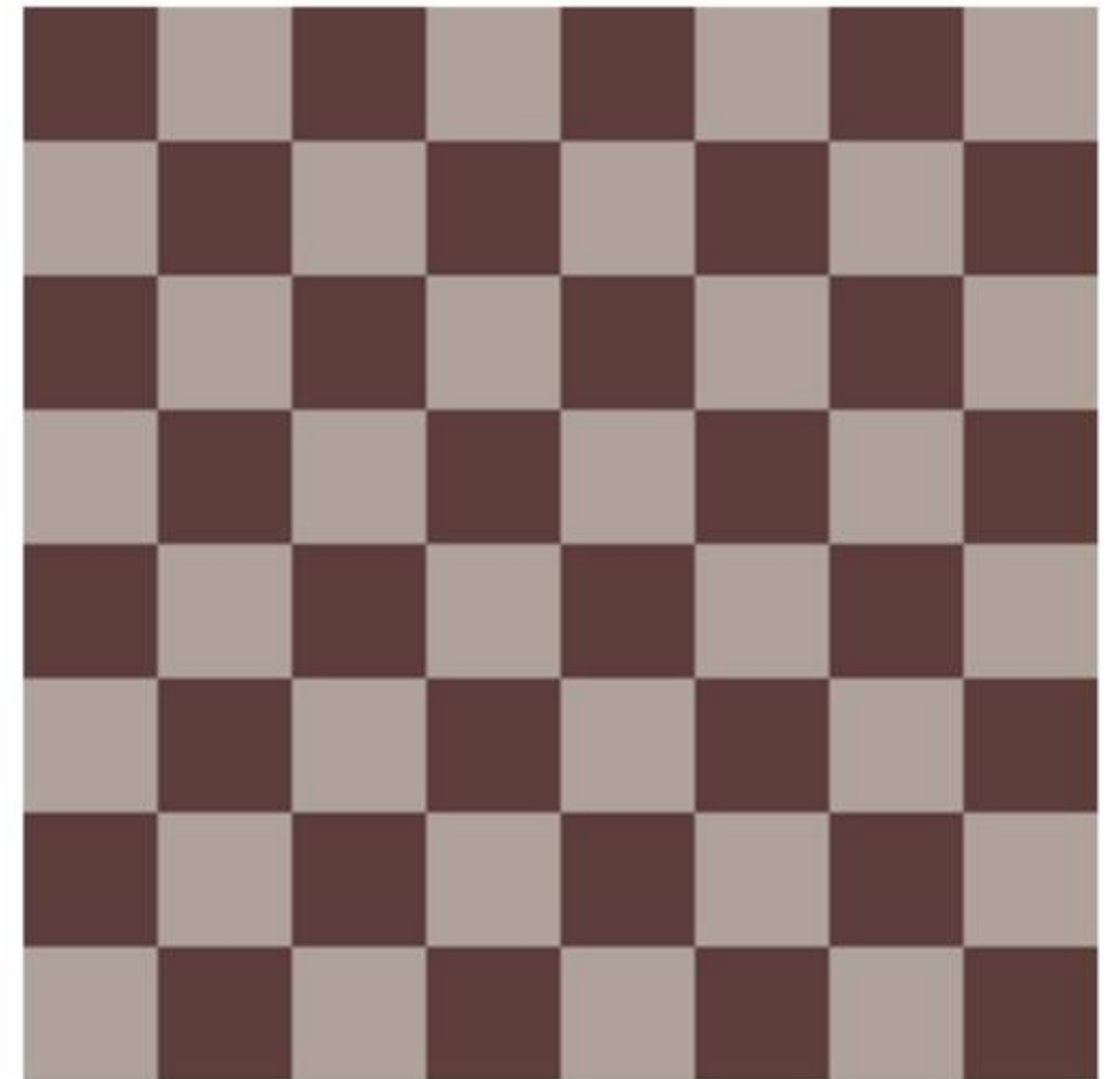
¿Por qué reconstruir un renderizador desde cero?

/main.rs

output.ppm

```
fn main() {  
    println!("P3\n8 8\n255");  
    for y in 0..8 {  
        for x in 0..8 {  
            let (r, g, b) = if (x + y) % 2 == 0 {  
                (94, 62, 58)  
            } else {  
                (175, 162, 154)  
            };  
            println!("{r} {g} {b}");  
        }  
    }  
}
```

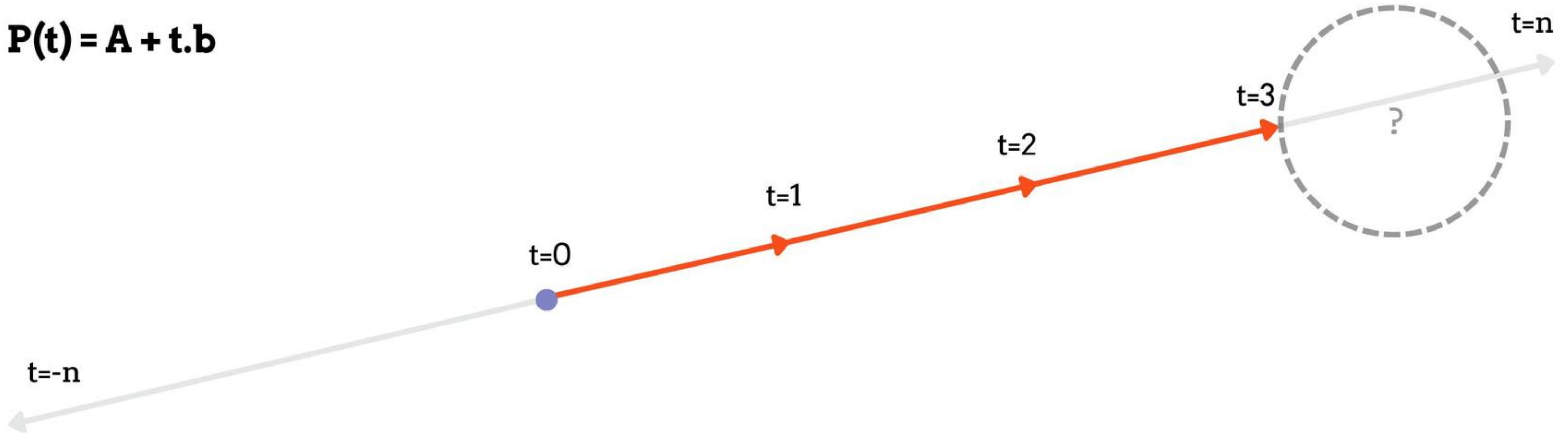
\$ cargo run > output.ppm



Path Tracer

¿Qué es un rayo?

$$\mathbf{P}(t) = \mathbf{A} + t \cdot \mathbf{b}$$

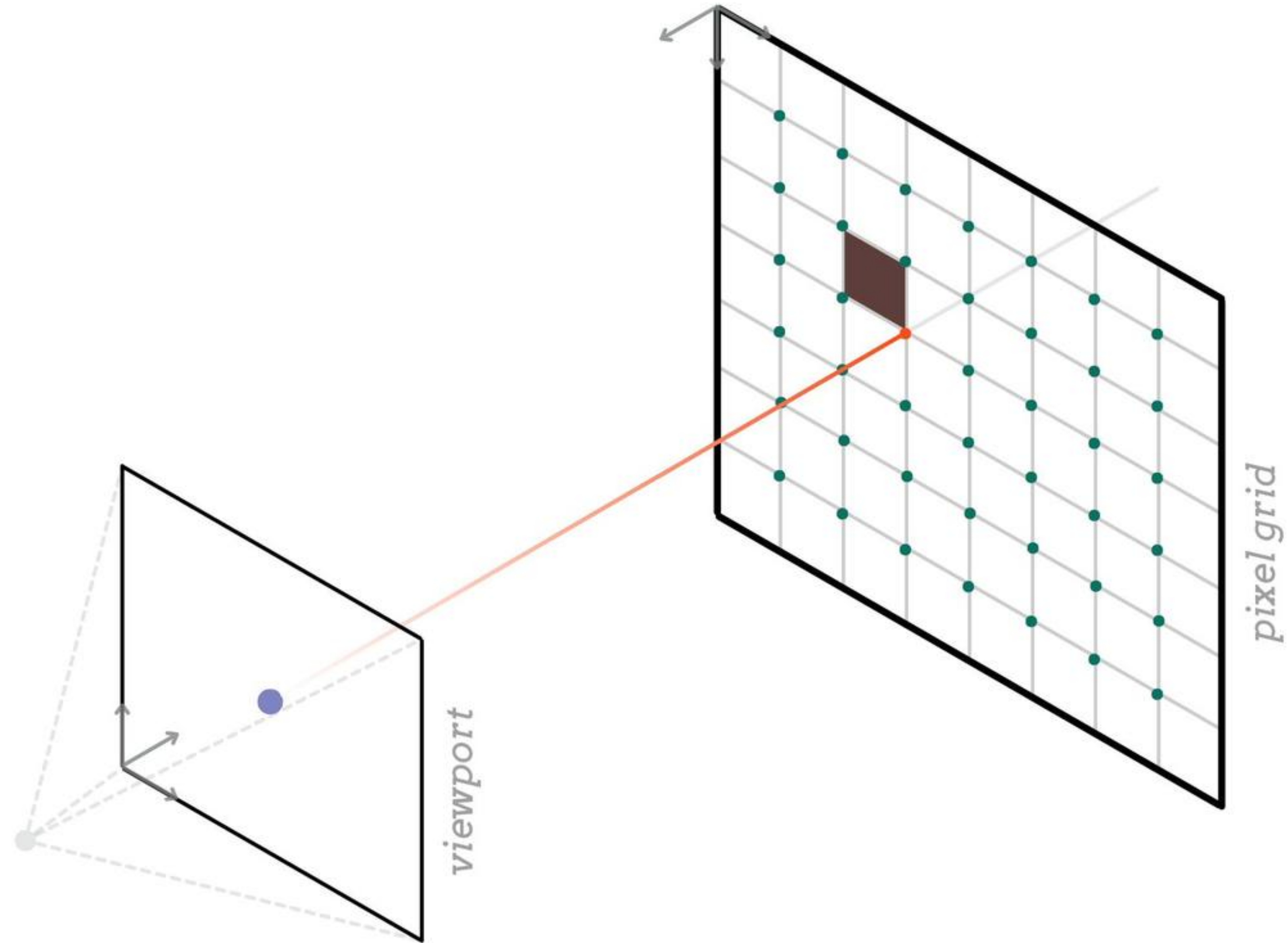


- A es el **origen** del rayo
- ▶ b es la **dirección** del rayo

Con distintos t , cambia la magnitud de $t \cdot \mathbf{b}$, moviendo P a lo largo del rayo

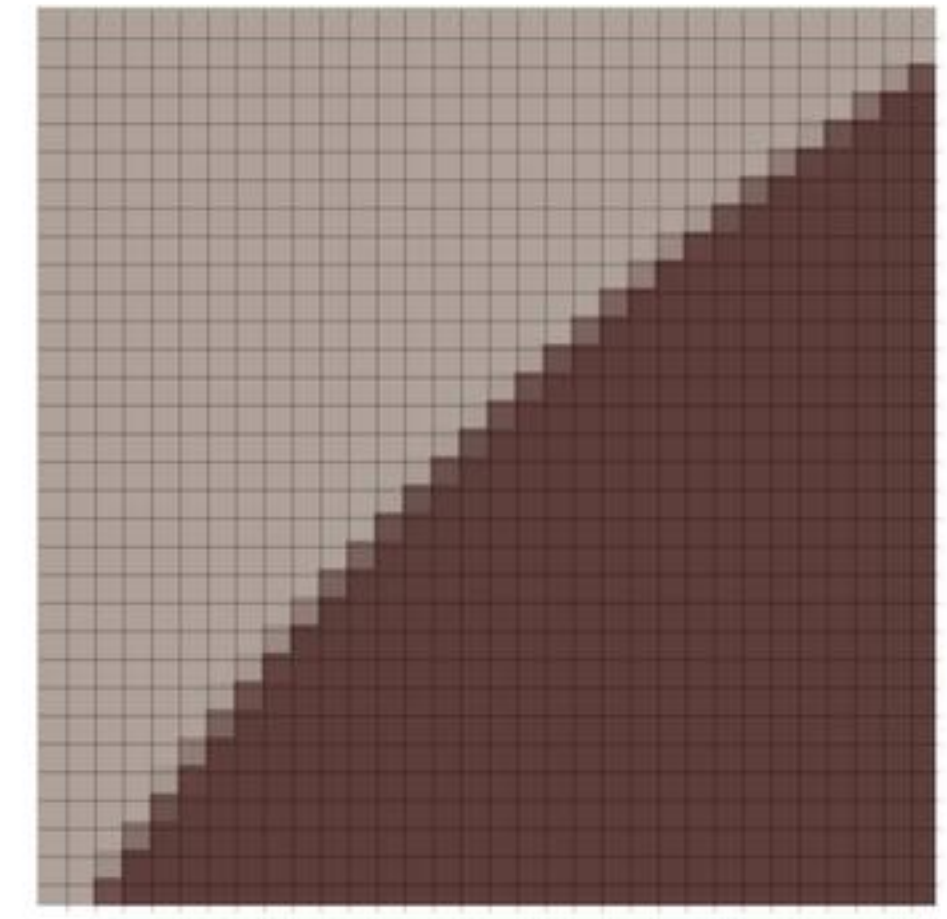
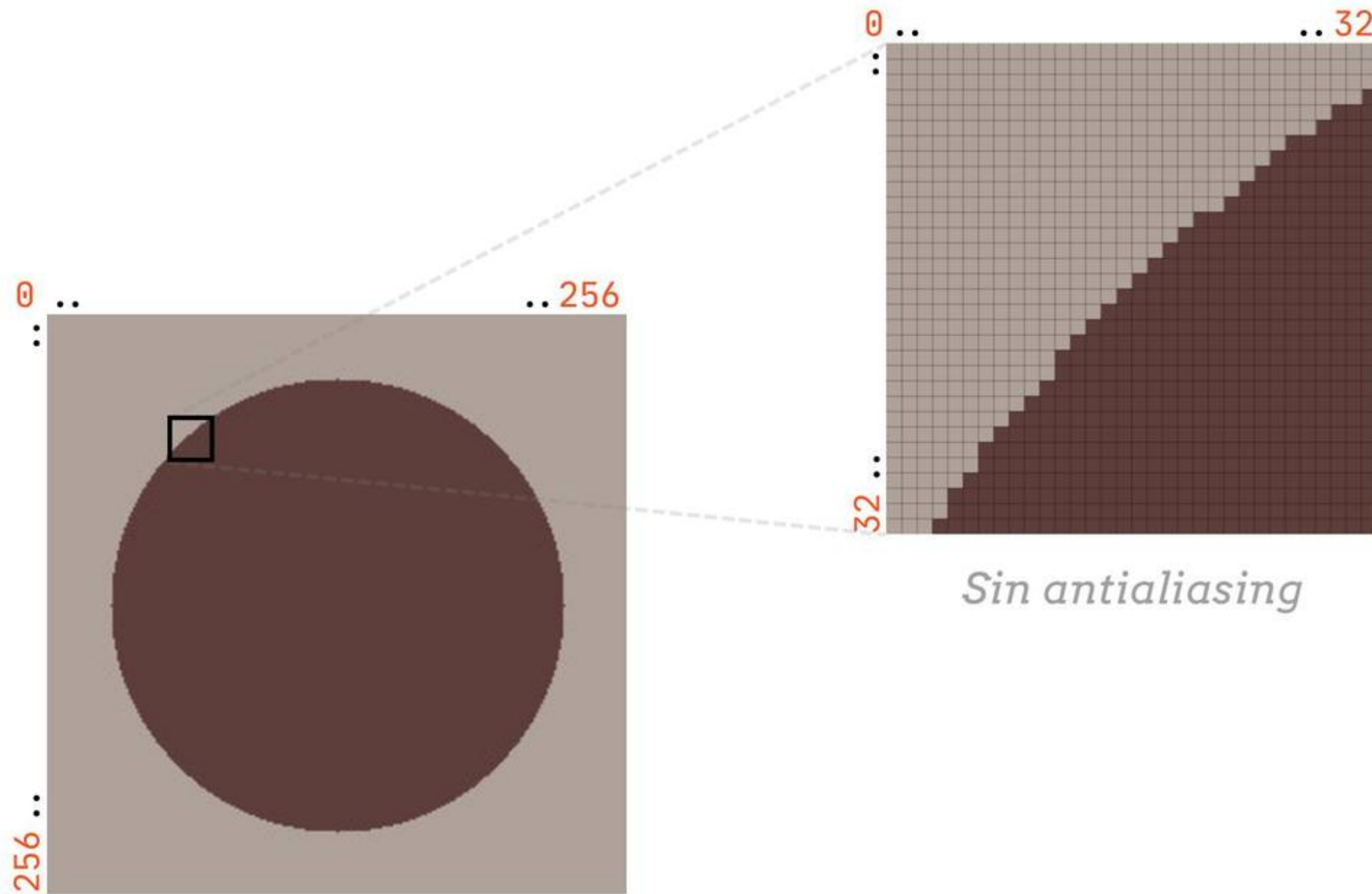
Path Tracer

Viewport y pixel grid



Path Tracer

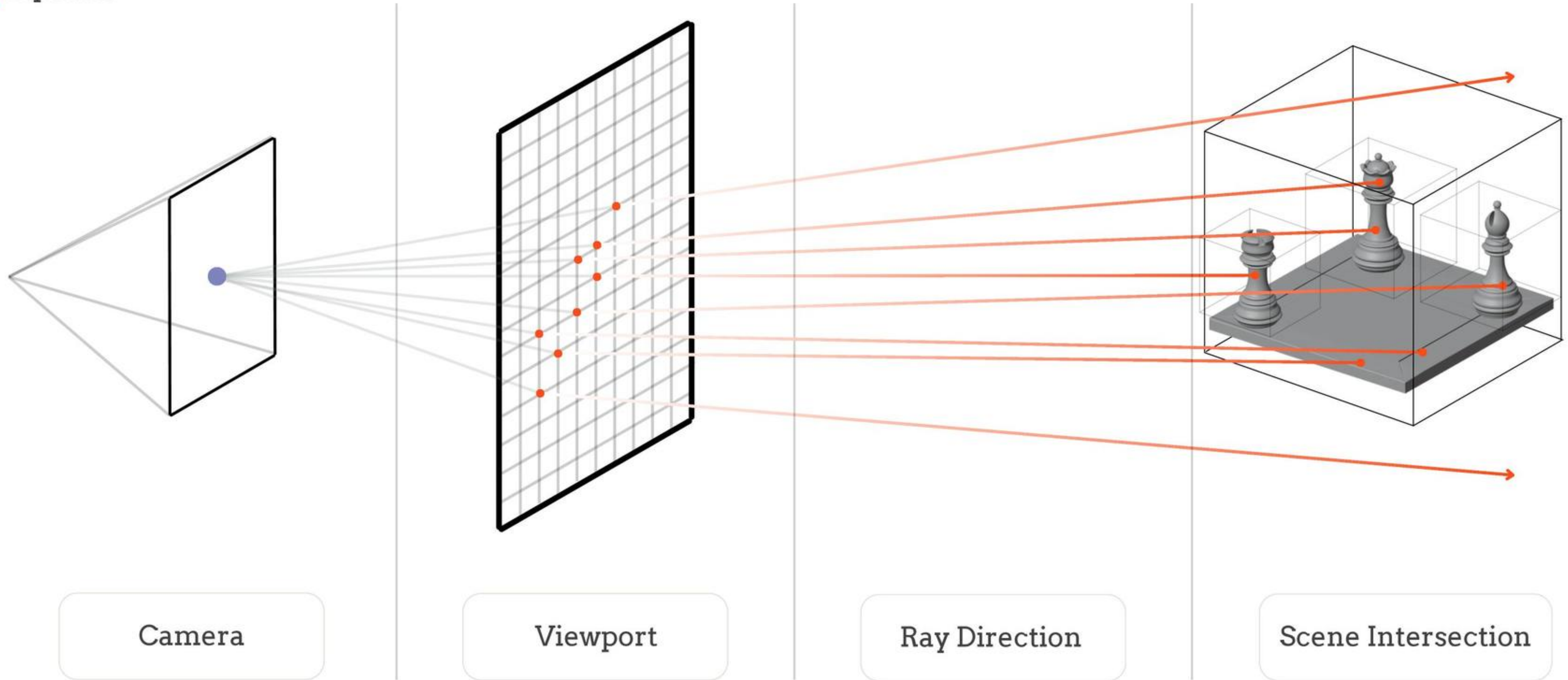
¿Qué es realmente un pixel?



Antialiasing
16 muestras por pixel

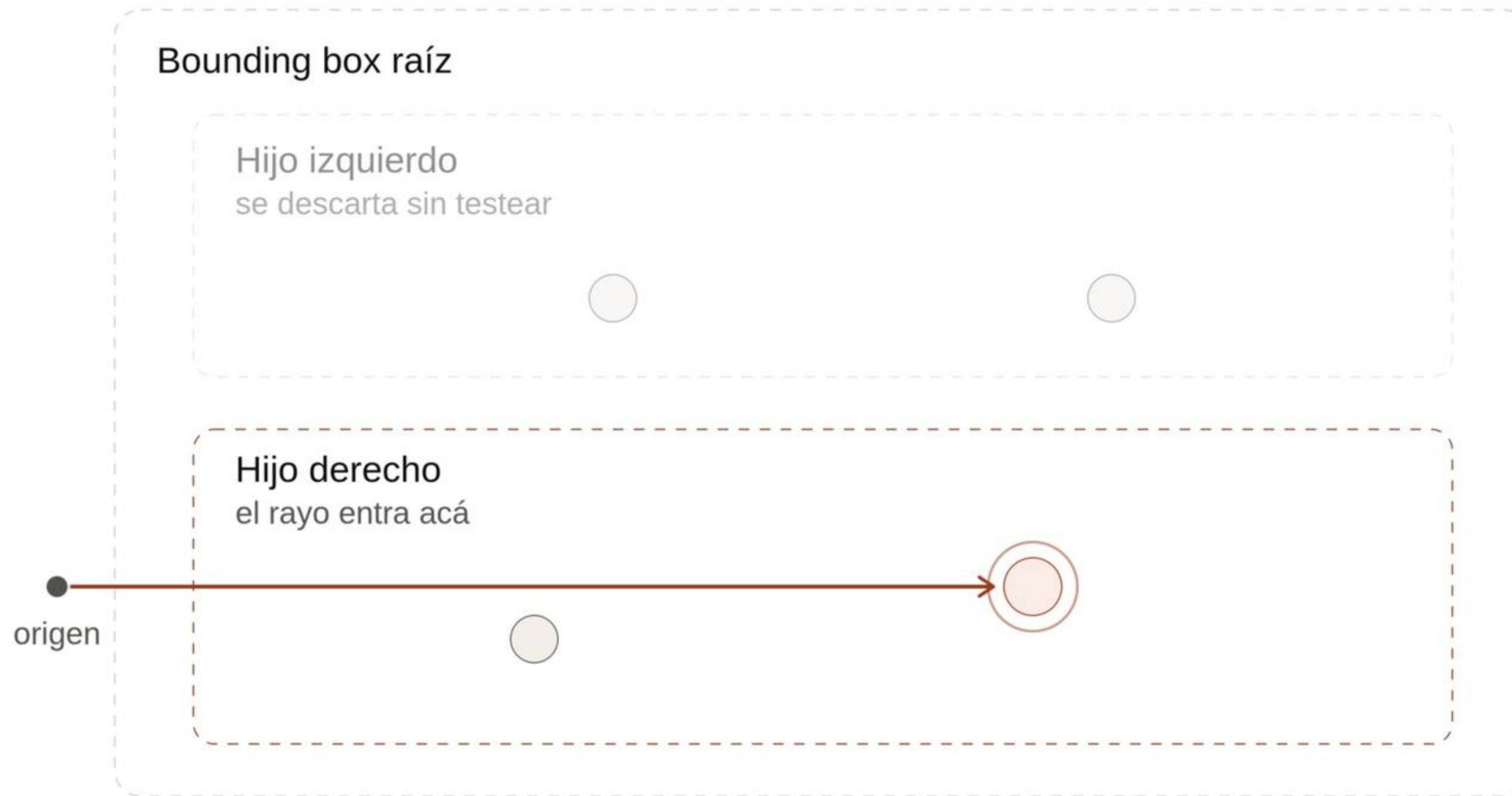
Path Tracer

Pipeline



Bounding Volume Hierarchy

¿Cómo evito testear cada rayo contra cada objeto?



Eleccion del Lenguaje

¿Por qué Rust y no C++?

C++ y similares

Rust

Rendimiento

Cercano al hardware

Equivalente, mayor tiempo de compilación

Portabilidad

Multiplataforma

Multiplataforma, vía LLVM

Estándar

Consolidado en la industria

En crecimiento pero en foco

Seguridad

Comportamientos indefinidos silenciosos

Verificada en compilación
(Safe Rust, sin interior mutability)

Toolchain

Sin toolchain unificado

Cargo

Memoria

Gestión manual

Sin tracing GC (Rc/Arc dan un GC simple)

Eleccion del Lenguaje

¿Por qué Rust y no C++?

C++ y similares

Rust

Rendimiento

Cercano al hardware

Equivalente, mayor tiempo de compilación

Portabilidad

Multiplataforma

Multiplataforma, vía LLVM

Estándar

Consolidado en la industria

En crecimiento pero en foco

Seguridad

Comportamientos indefinidos silenciosos

Verificada en compilación
(Safe Rust, sin interior mutability)

Toolchain

Sin toolchain unificado

Cargo

Memoria

Gestión manual

Sin tracing GC (Rc/Arc dan un GC simple)

Eleccion del Lenguaje

¿Por qué Rust y no C++?

C++ y similares

Rust

Rendimiento

Cercano al hardware

Equivalente, mayor tiempo de compilación

Portabilidad

Multiplataforma

Multiplataforma, vía LLVM

Estándar

Consolidado en la industria

En crecimiento pero en foco

Seguridad

Comportamientos indefinidos silenciosos

Verificada en compilación
(Safe Rust, sin interior mutability)

Toolchain

Sin toolchain unificado

Cargo

Memoria

Gestión manual

Sin tracing GC (Rc/Arc dan un GC simple)

Eleccion del Lenguaje

¿Por qué Rust y no C++?

C++ y similares

Rust

Rendimiento

Cercano al hardware

Equivalente, mayor tiempo de compilación

Portabilidad

Multiplataforma

Multiplataforma, vía LLVM

Estándar

Consolidado en la industria

En crecimiento pero en foco

Seguridad

Comportamientos indefinidos silenciosos

Verificada en compilación
(Safe Rust, sin interior mutability)

Toolchain

Sin toolchain unificado

Cargo

Memoria

Gestión manual

Sin tracing GC (Rc/Arc dan un GC simple)

Eleccion del Lenguaje

¿Por qué Rust y no C++?

C++ y similares

Rust

Rendimiento

Cercano al hardware

Equivalente, mayor tiempo de compilación

Portabilidad

Multiplataforma

Multiplataforma, vía LLVM

Estándar

Consolidado en la industria

En crecimiento pero en foco

Seguridad

Comportamientos indefinidos silenciosos

Verificada en compilación
(Safe Rust, sin interior mutability)

Toolchain

Sin toolchain unificado

Cargo

Memoria

Gestión manual

Sin tracing GC (Rc/Arc dan un GC simple)

Eleccion del Lenguaje

¿Por qué Rust y no C++?

C++ y similares

Rust

Rendimiento

Cercano al hardware

Equivalente, mayor tiempo de compilación

Portabilidad

Multiplataforma

Multiplataforma, vía LLVM

Estándar

Consolidado en la industria

En crecimiento pero en foco

Seguridad

Comportamientos indefinidos silenciosos

Verificada en compilación
(Safe Rust, sin interior mutability)

Toolchain

Sin toolchain unificado

Cargo

Memoria

Gestión manual

Sin tracing GC (Rc/Arc dan un GC simple)

Estructuras

Clase principal de componente lineal

vec3.h

```
1  class vec3 {
2      public:
3          ^^^^^^^^ modificador de acceso
4          double e[3];
5          ^^^^^^^^^^^^^^^^^^ declaración de atributo
6          vec3() : e{0,0,0} {}
7
8          double x(&this) const {
9              return this->e[0];
10         }
11         double y() const { return e[1]; }
12         double z() const { return e[2]; }
13     }
```

coord.rs

```
| struct Coord{pub e: [f32; 3]};
| impl Coord { ^^^^^^^^ declaración de atributo y mod
|     pub fn new3(x: f32, y: f32, z: f32) -> Self
|     {
|         Self([x, y, z])
|     }
|
|     pub fn x(&self) -> f32 {
|         self.e[0]
|     }
|     pub fn y(&self) -> f32 { self.e[1] }
|     pub fn z(&self) -> f32 { return self.e[2]; }
| }
```

Estructuras

Clase principal de componente lineal

vec3.h

```
1  class vec3 {
2      public:
3
4      double e[3];
5
6      vec3() : e{0,0,0} {}
7      ^^^^^^^ constructor vacío
8      double x(&this) const {
9          return this->e[0];
10     }
11     double y() const { return e[1]; }
12     double z() const { return e[2]; }
13 }
```

coord.rs

```
| struct Coord{pub e: [f32; 3]};
| impl Coord {
|     pub fn new3(x: f32, y: f32, z: f32) -> Self
|     {
|         ^^^^^^ constructor por convención "new"
|         Self([x, y, z])
|     }
|
|     pub fn x(&self) -> f32 {
|         self.e[0]
|     }
|     pub fn y(&self) -> f32 { self.e[1] }
|     pub fn z(&self) -> f32 { return self.e[2]; }
| }
```

Colecciones

Cómo le digo al compilador que necesito algo en el heap?

```
1 struct Primitive;
2
3 error[E0072]: recursive type `HittableList` has infinite size
4 enum HittableList {
5     Elem(Primitive, HittableList),
6     Empty,           ----- recursive without indirection
7 }

4 help: insert some indirection (e.g., a `Box`, `Rc`, or `&`) to break the cycle
5     Elem(Primitive, Box<HittableList>),
                   ++++++          +
```



too-many-lists

Colecciones

Otro ejemplo de uso de Box

```
1  pub enum Bvh {  
2      Empty,  
3      Node {  
4          bbox: Aabb,  
5          left: Box<Bvh>,  
6          right: Box<Bvh>,  
7      },  
8      Leaf(Shared<dyn Primitive>),  
9  }
```

?

pila

?

heap

Colecciones

¿Cómo está representado realmente un árbol?

```
1 let root = Bvh::Node {  
2   bbox: aabb,  
3   left: Box::new(l_bvh),  
4   right: Box::new(r_bvh),  
5 };
```

?

pila

```
Node {  
  bbox: Aabb,  
  left: Box<Bvh>,  
  right: Box<Bvh>,  
};
```



heap

```
Leaf(  
  Shared<dynPrimitive>  
)
```



Estructuras

Clase principal de componente lineal

vec3.h

```
1  class vec3 {
2      public:
3
4      double e[3];
5
6      vec3() : e{0,0,0} {}
7
8      double x(&this) const {
9          return this->e[0];
10     }          ^^^^ parámetro implícito
11     double y() const { return e[1]; }
12     double z() const { return e[2]; }
13 }
```

coord.rs

```
| struct Coord{pub e: [f32; 3]};
| impl Coord {
|     pub fn new3(x: f32, y: f32, z: f32) -> Self
|     {
|         Self([x, y, z])
|     }
|
|     pub fn x(&self) -> f32 {
|         self.e[0]
|     } ^^^^ parámetro explícito
|     pub fn y(&self) -> f32 { self.e[1] }
|     pub fn z(&self) -> f32 { return self.e[2]; }
| }
```

vec3.h

```

1  class vec3 {
2      public:
3
4          double e[3];
5
6          vec3() : e{0,0,0} {}
7
8          double x(&this) const {
9              return this.e[0];
10         }
11         double y() const { return e[1]; }
12         double z() const { return e[2]; }
13     }

```

coord.rs

```
struct Coord{pub e: [f32; 3]};
impl Coord {
    pub fn new3(x: f32, y: f32, z: f32) -> Self
    {
        Self([x, y, z])
    }

    pub fn x(&self) -> f32 {
        self.e[0]
    }

    pub fn y(&self) -> f32 { self.e[1] }
    pub fn z(&self) -> f32 { return self.e[2]}
}
return optional ^^^^^^^
```

Estructuras

Type aliases — no crean tipos nuevos, solo apodos

color.h

```
using color = vec3;
```

≡

vec3.h

```
using point3 = vec3;
```

≡

vec3.h

```
class vec3 { .. }
```

ray.h

```
1  class ray {
2      point3 at(double t) const {
3          return orig + t*dir;
4      }
5      ...
6
7      private:
8          point3 orig;
9          vec3 dir;
10 }
```

_.cpp

```
|  color o = color(0,0,0);
|  point3 d = vec3(1,0,0);
|
|  ray r = ray(d, o);
|  r.at(3.0);
|      ^^^^^^^^^^^^^^^ vec3 + 3.0*color?
```



Some people don't like this because it doesn't prevent you from doing something silly, **like subtracting a position from a color**. They have a good point, but we're going to always take the "less code" route when not obviously wrong



New Type

Estructuras con un solo componente para obtener **strict type checking**

```
1 struct UnitVec3(Vec3);    ≡
2 struct Vec3(Coord);       ≡
3 struct Point3(Coord);     ≡
4 struct Coord([f32; 3]);
5
6 error[E0308]: mismatched types
7 fn main() {
8     let v1: Vec3 = Coord([0.0, 0.0, 0.0]);
9 }
   ---- ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ expected `Vec3`, found `Coord`
   |
   expected due to this

7 help: try wrapping the expression in `Vec3`
8     let v1: Vec3 = Vec3(Coord([0.0, 0.0, 0.0]));
                      ++++++ +
```

Zero cost abstractions

Comparando el layout

```
1  #[test]
2  fn newtypes_have_same_layout_as_raw() {
3      assert_eq!(size_of::<Coord>(), size_of::<[f32; 3]>());
4      assert_eq!(align_of::<Coord>(), align_of::<[f32; 3]>());
5
6      assert_eq!(size_of::<Vec3>(), size_of::<[f32; 3]>());
7      assert_eq!(align_of::<Vec3>(), align_of::<[f32; 3]>());
8
9      assert_eq!(size_of::<UnitVec3>(), size_of::<[f32; 3]>());
10     assert_eq!(align_of::<UnitVec3>(), align_of::<[f32; 3]>());
11
12     assert_eq!(size_of::<Point3>(), size_of::<[f32; 3]>());
13     assert_eq!(align_of::<Point3>(), align_of::<[f32; 3]>());
14 }
```

Tipo		Size	Align
[f32; 3]		12B	4B
Coord	✓	12B	4B
Vec3	✓	12B	4B
UnitVec3	✓	12B	4B
Point3	✓	12B	4B

Zero cost abstractions

Comparando el assembly generado

Point3(Coord([1.0, 2.0, 3.0]))

```
1  .LC:
2  .long 0x3f800000 1.0f
3  .long 0x40000000 2.0f
4
5  newtypes:
6  movsd xmm0, qword ptr [rip + .LC]
7  xmm0 = {1.0, 2.0}
8  movsd qword ptr [rsp - 16], xmm0
9  p[0..2] = {1.0, 2.0}
10 mov dword ptr [rsp - 8], 1077936128
11 p[2] = 3.0
```

[1.0, 2.0, 3.0]

```
| .LC:
| .long 0x3f800000
| .long 0x40000000
|
| raw:
| movsd xmm0, qword ptr [rip + .LC]
|
| movsd qword ptr [rsp - 16], xmm0
|
| mov dword ptr [rsp - 8], 1077936128
|
```

\$ rustc -C opt-level=3

Polimorfismo

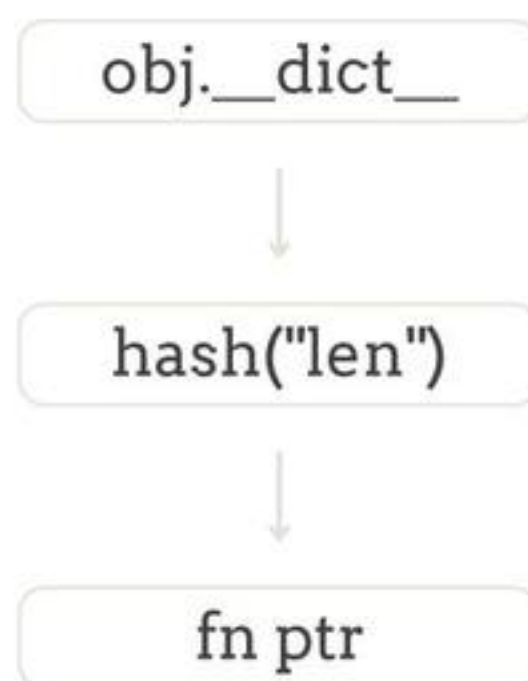
Tres formas de resolverlo

runtime · hash lookup

Lenguajes dinámicos

Python, Ruby, JavaScript...

Llamar a un método es buscar un string en un hashmap, en cada llamada.



runtime · vtable

Dispatch dinámico, tipado estático

C++ (virtual), Java, Swift, Rust (dyn Trait)...

Cada objeto guarda un puntero a una tabla fija de punteros a función (vtable), armada una vez por clase. Llamar a un método es un salto indirecto.



compile-time · monomorphization

Genéricos y templates

Rust (generics), C++ (templates)...

El compilador conoce el tipo concreto en cada sitio de uso y genera una copia especializada del código por cada instancia. Sin indirección en runtime, pero el binario crece.



Polimorfismo

Tres formas de resolverlo

runtime · hash lookup

Lenguajes dinámicos

Python, Ruby, JavaScript...

Llamar a un método es buscar un string en un hashmap, en cada llamada.



runtime · vtable

Dispatch dinámico, tipado estático

C++ (virtual), Java, Swift, Rust (dyn Trait)...

Cada objeto guarda un puntero a una tabla fija de punteros a función (vtable), armada una vez por clase. Llamar a un método es un salto indirecto.



compile-time · monomorphization

Genéricos y templates

Rust (generics), C++ (templates)...

El compilador conoce el tipo concreto en cada sitio de uso y genera una copia especializada del código por cada instancia. Sin indirección en runtime, pero el binario crece.



Polimorfismo

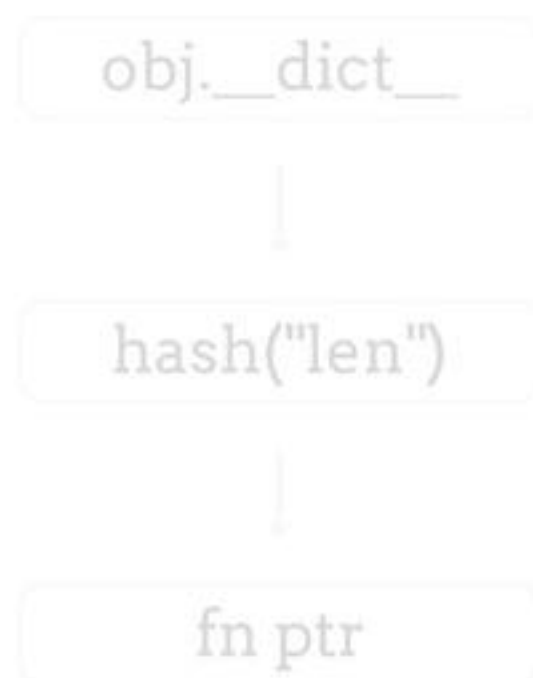
Tres formas de resolverlo

runtime · hash lookup

Lenguajes dinámicos

Python, Ruby, JavaScript...

Llamar a un método es buscar un string en un hashmap, en cada llamada.



runtime · vtable

Dispatch dinámico, tipado estático

C++ (virtual), Java, Swift, Rust (dyn Trait)...

Cada objeto guarda un puntero a una tabla fija de punteros a función (vtable), armada una vez por clase. Llamar a un método es un salto indirecto.

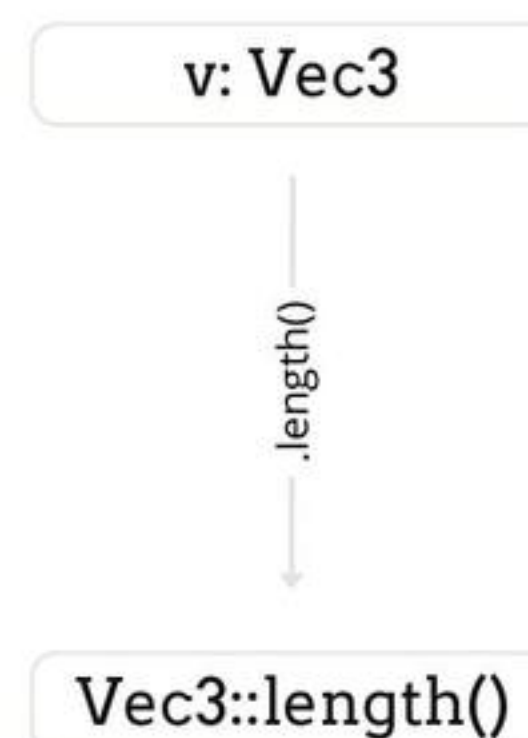


compile-time · monomorphization

Genéricos y templates

Rust (generics), C++ (templates)...

El compilador conoce el tipo concreto en cada sitio de uso y genera una copia especializada del código por cada instancia. Sin indirección en runtime, pero el binario crece.



Mutabilidad

Cómo Rust previene aliasing violations

inherited

```
1 let mut x = Sphere::new(5.0);
2
3 let r1 = &x.radius;
4           ----- immutable borrow occurs here
5 let r2 = &mut x.radius;
6           ^^^^^^^^^^^^^^^^^ mutable borrow occurs here
7 println!("{}", r1);
8           -- immutable borrow later used here
```

interior

```
let mut x = RefCell::new(Sphere::new(5.0));

let r1 = x.borrow();

let r2 = x.borrow_mut();

println!("{}", r1);
```

Mutabilidad

Cómo Rust previene aliasing violations

inherited

```
1 let mut x = Sphere::new(5.0);
2
3 let r1 = &x.radius;
4
5 let r2 = &mut x.radius;
6
7 println!("{}", r1);
```

```
$ cargo build
```

```
error[E0502]: cannot borrow `x.radius` as mutable
because it is also borrowed as immutable
```

interior

```
let mut x = RefCell::new(Sphere::new(5.0));

let r1 = x.borrow();

let r2 = x.borrow_mut();

println!("{}", r1);
```

```
$ cargo run
```

```
thread 'main' (17047) panicked at src/main.rs:7:20:
RefCell already borrowed
```

mismo error, dos momentos distinto

Escena de prueba

Configuración usada para todos los benchmarks

Parámetros de render

 Resolución
600x600

 Samples/px
50

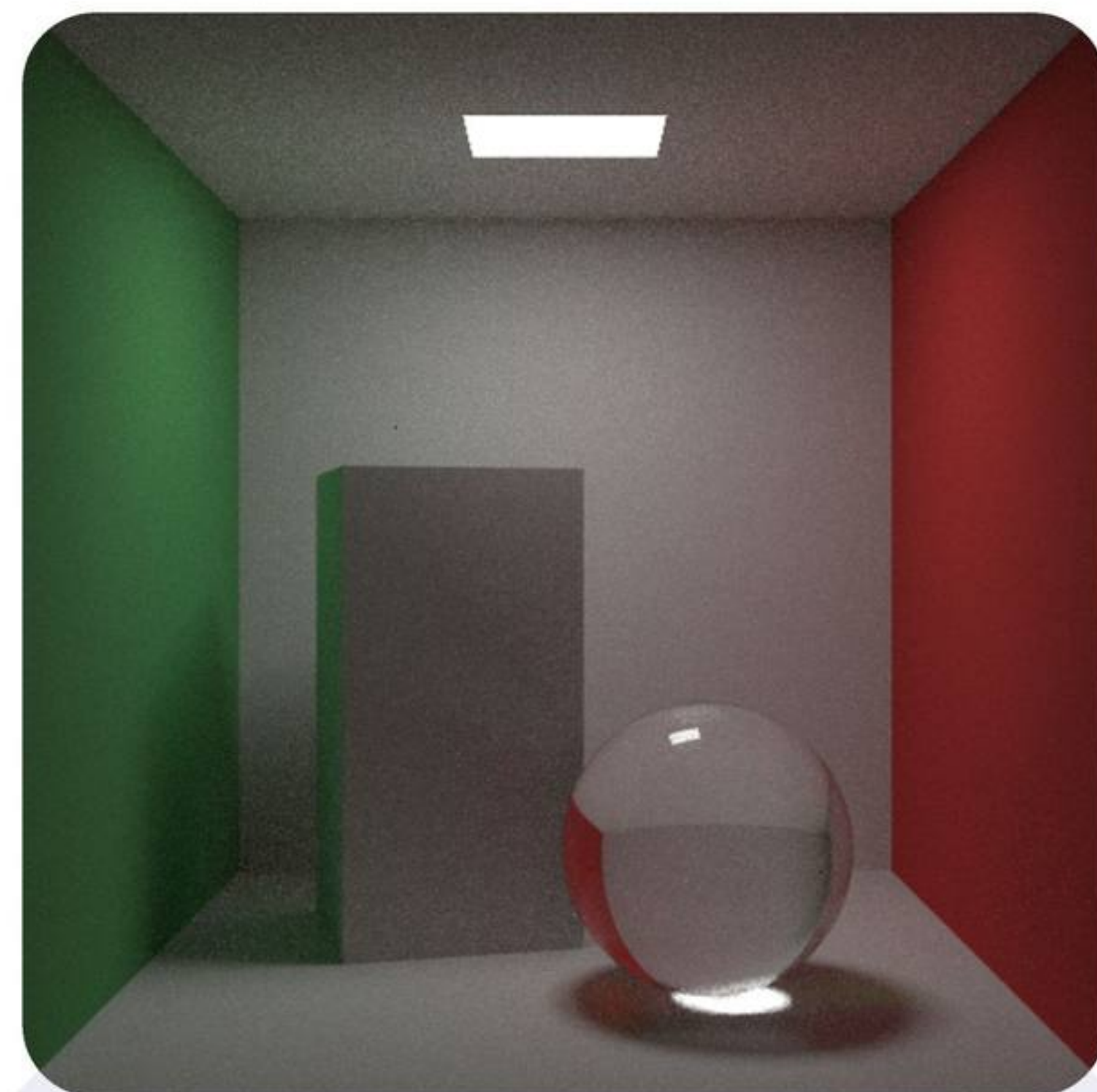
 Profund. máx
50

Hardware

 CPU
AMD Ryzen 5 9600X
3.9 GHz

 GPU

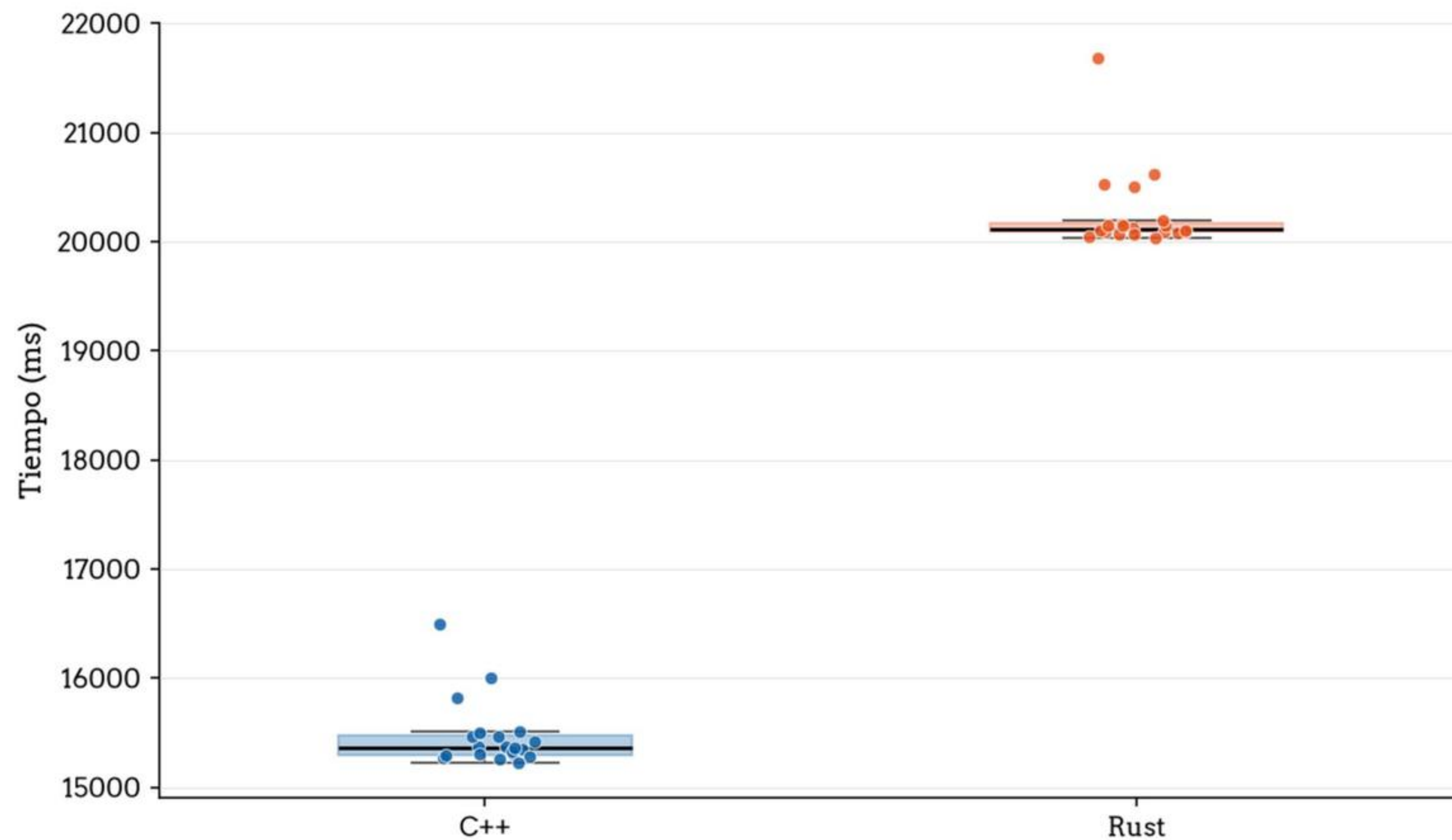
 RAM
32.0 GB
6000 MT/s



Cornell box

Benchmarks

C++ vs Rust — single-threaded



Rust

- min: 20037.895 ms
- **median: 20115.715 ms**
- mean: 20251.919 ms
- p95: 20618.851 ms

C++

- min: 15224.763 ms
- **median: 15366.298 ms**
- mean: 15468.999 ms
- p95: 15997.371 ms

Rust optimizado

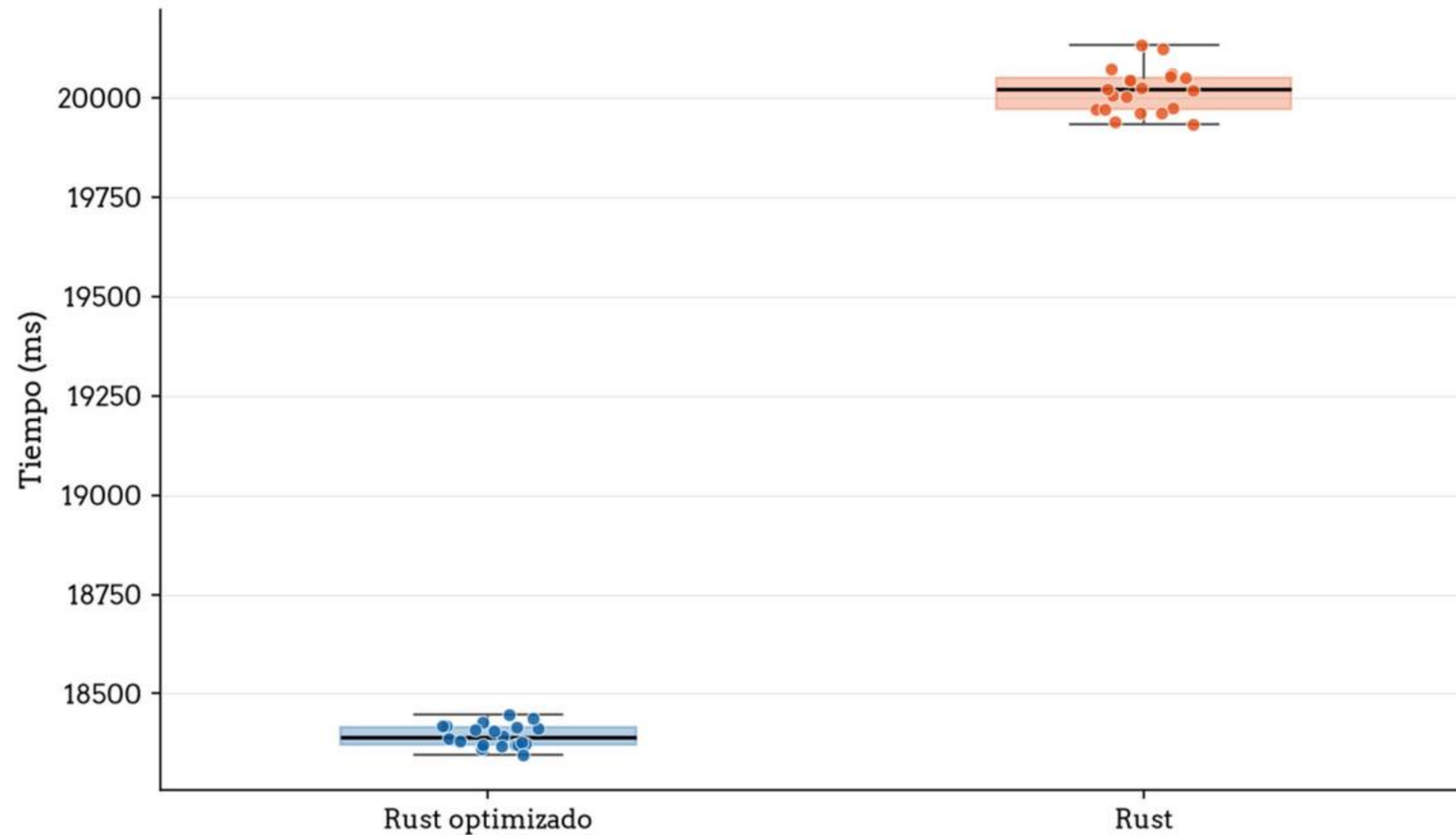
Cargo.toml usado para el profile de release

```
1  [profile.release]
2  opt-level = 3
3  lto = "fat"
4  codegen-units = 1
5  panic = "abort"
6  incremental = false
7  debug = false
8  strip = "symbols"
9
10 [build]
11 rustflags = ["-C", "target-cpu=native"]
```

target-cpu=native ata el binario a esta CPU: no es portable a otras maquinas

Benchmarks

Rust optimizado vs Rust



Rust

- min: 19934.708 ms
- **median: 20021.325 ms**
- mean: 20018.749 ms
- p95: 20123.464 ms

Rust optimizado

- min: 18347.054 ms
- **median: 18390.472 ms**
- mean: 18395.465 ms
- p95: 18439.583 ms

Benchmarks

SIMD - una instrucción, varios datos a la vez

```
1 #[repr(C, align(16))]  
2 #[derive(Clone, PartialEq)]  
3 struct Coord([f32; 4]);
```

registro SIMD de 128 bits

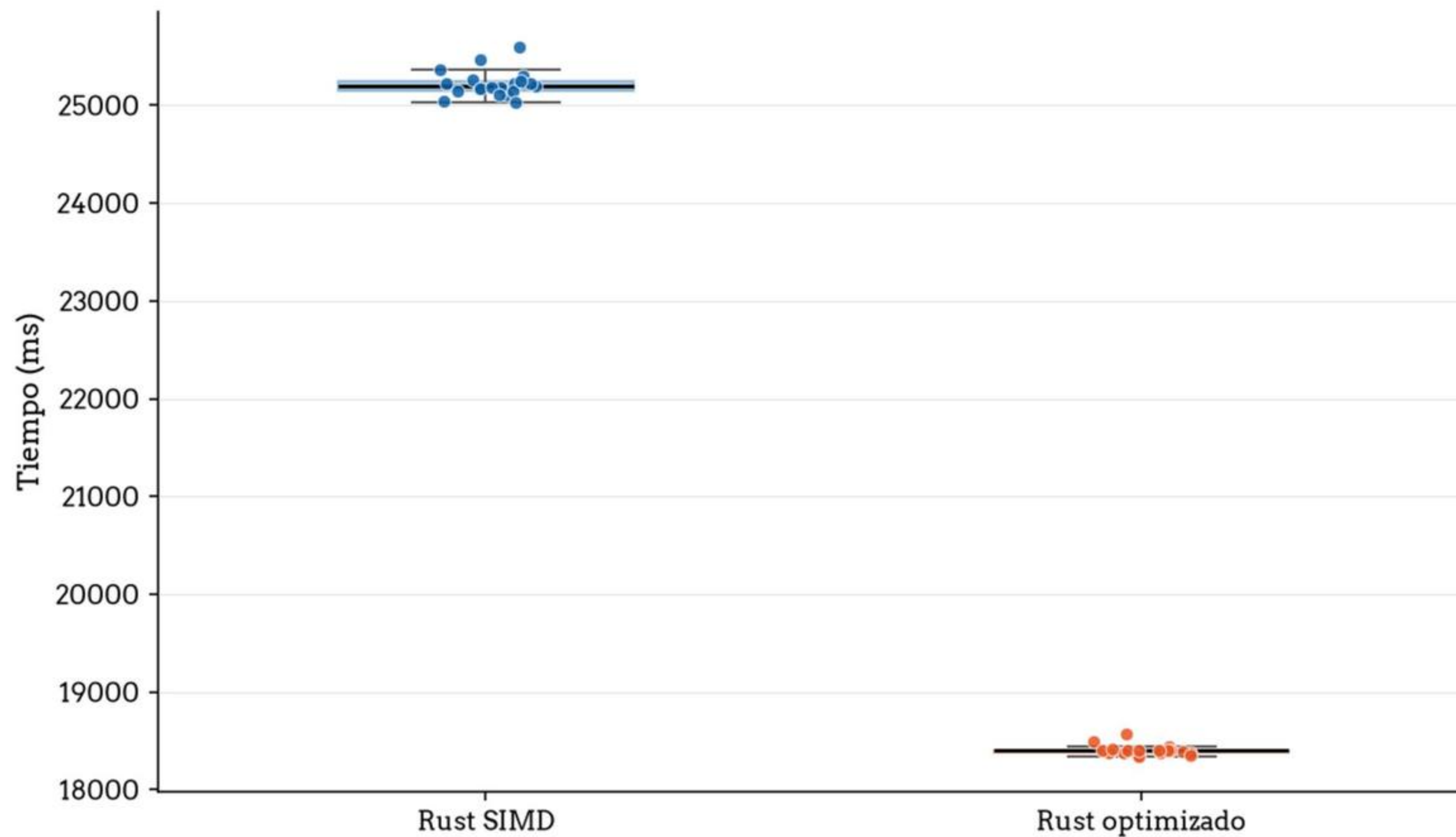


1 instrucción, 4 lanes — pero 1 lane vacío

25 % del struct es relleno para llegar a 16 bytes alineados

Benchmarks

Rust SIMD vs Rust optimizado



Rust SIMD

- min: 25028.935 ms
- **median: 25195.030 ms**
- mean: 25220.854 ms
- p95: 25469.998 ms

Rust optimizado

- min: 18342.317 ms
- **median: 18397.208 ms**
- mean: 18403.514 ms
- p95: 18497.597 ms

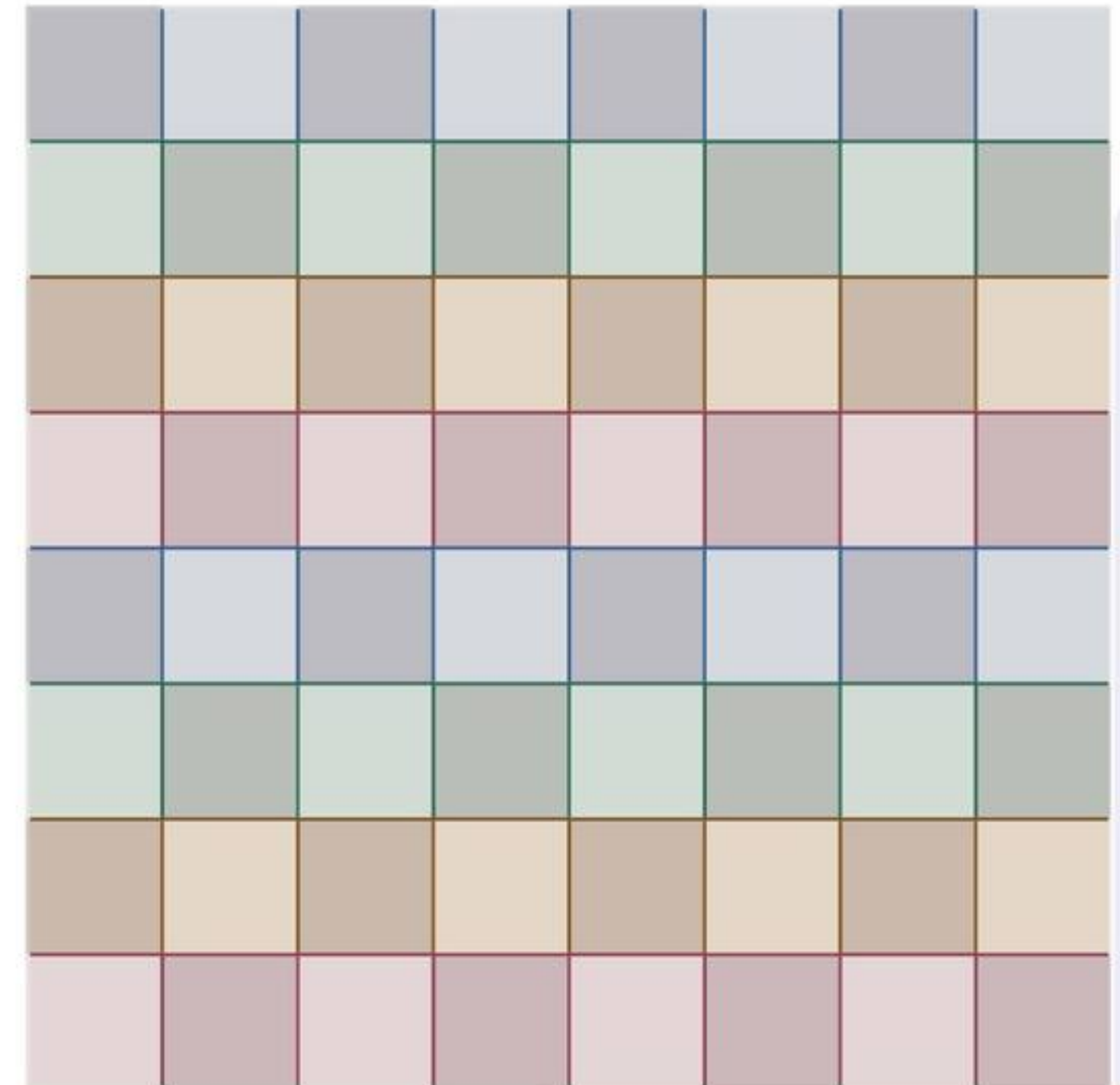
Benchmarks

Rayon — paralelizando el render entre nucleos

Rayon divide el rango en bloques
y los reparte entre un pool de threads.

Con 4 nucleos, hasta 4 filas
se calculan al mismo tiempo.

-  nucleo 1 — filas 0, 4
-  nucleo 2 — filas 1, 5
-  nucleo 3 — filas 2, 6
-  nucleo 4 — filas 3, 7



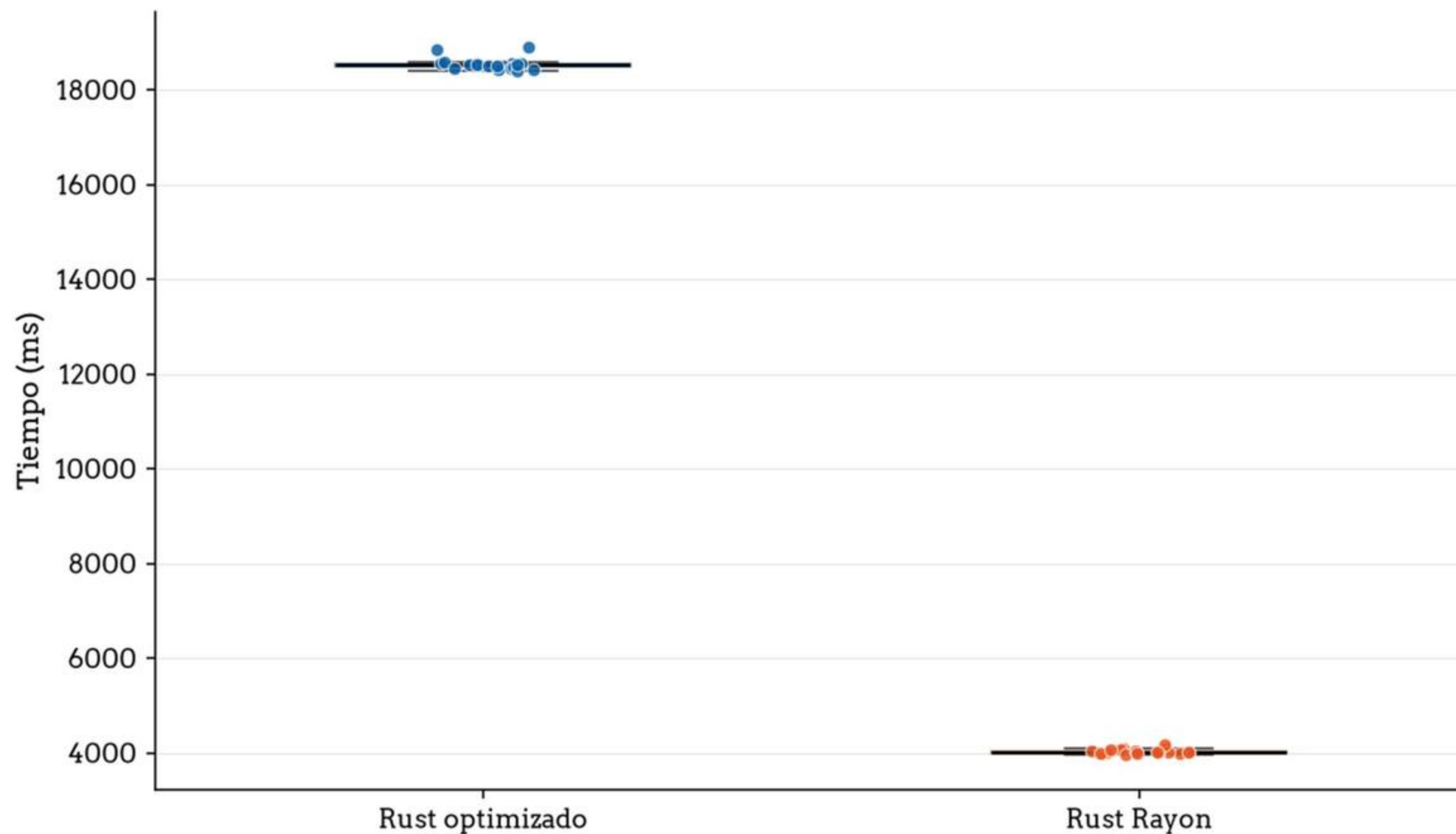
Benchmarks

Rayon — el cambio es una linea

```
1 pub fn render(&self, world: &dyn Hittable, lights: &dyn Primitive) → Vec<f32>
2 {
3     #[cfg(feature = "parallel")]
4     let iter = (0..self.render.image_height).into_par_iter();
5
6     #[cfg(not(feature = "parallel"))]
7     let iter = (0..self.render.image_height).into_iter();
8
9     let rows: Vec<Vec<f32>> = iter.map(|j| {
10         ...
11         self.get_ray(i, j, s_i, s_j);
12         ...
13     }).collect();
```

Benchmarks

Rust optimizado vs Rust + Rayon



Rust optimizado

- min: 18400.283 ms
- median: 18524.589 ms
- mean: 18540.955 ms
- p95: 18842.196 ms

Rust Rayon

- min: 3967.427 ms
- **median: 4013.789 ms**
- mean: 4027.501 ms
- p95: 4086.839 ms

Demo Time!

```
$ cargo run --release
```



¿Me conviene aprender Rust?



¿Por qué este problema no se soluciona?

La ley de Moore aplica también para los equipos médicos

